

UNIVERSIDAD NACIONAL DE INGENIERÍA



PROGRAMACION II

INTEGRANTES:

Richard Josué Quezada Solano.

Marvin Francisco Pérez Hernández.

Michelle Alejandra Arce Vargas.

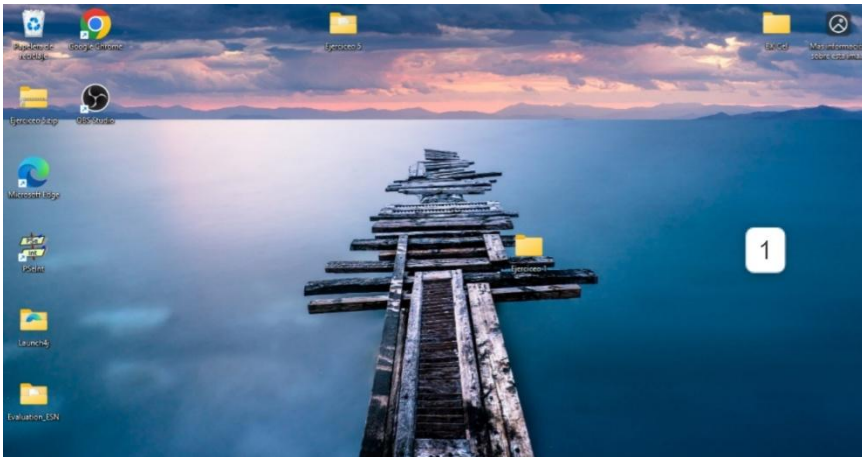
Sofía Alejandra Calero Huete.

Mauricio Miguel García M.

Profesor:

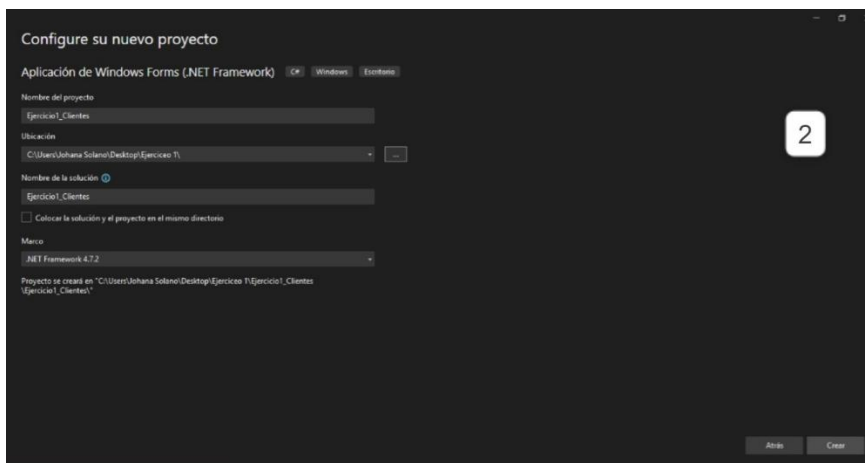
Ing. Abel Edmundo Marín Reyes

1) Ejercicio :



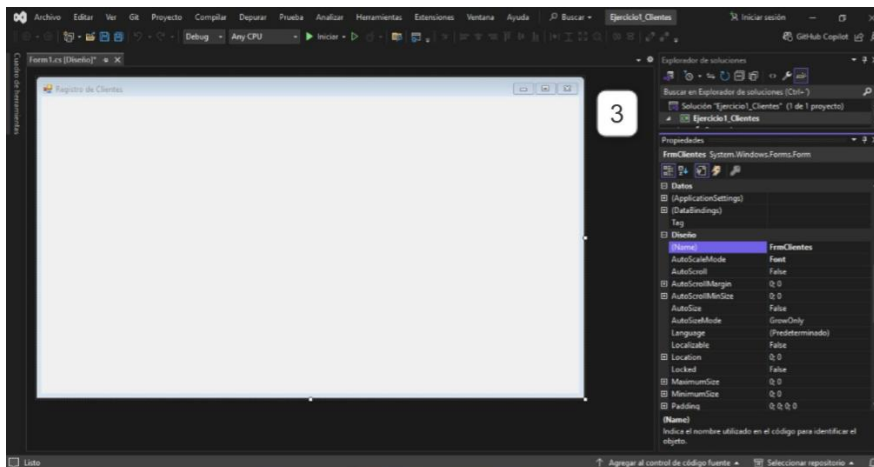
Creación de la carpeta del ejercicio

muestra la creación de la carpeta principal llamada Ejercicio 1 en el escritorio. Esta carpeta se utilizó para organizar todos los archivos correspondientes al desarrollo del sistema de registro de clientes.



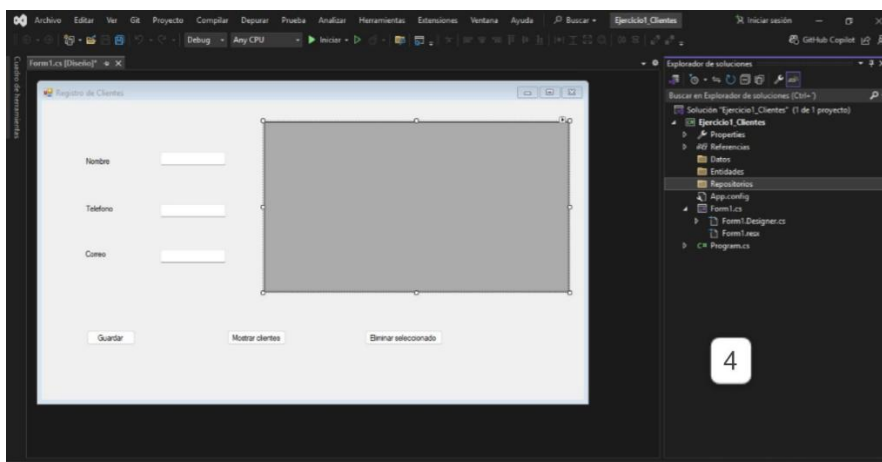
Configuración inicial del proyecto

observa la configuración del nuevo proyecto en Visual Studio. Se seleccionó una Aplicación de Windows Forms (.NET Framework), asignando como nombre del proyecto y de la solución Ejercicio1_Clientes. También se seleccionó el framework .NET Framework 4.7.2, que será utilizado para desarrollar la aplicación.



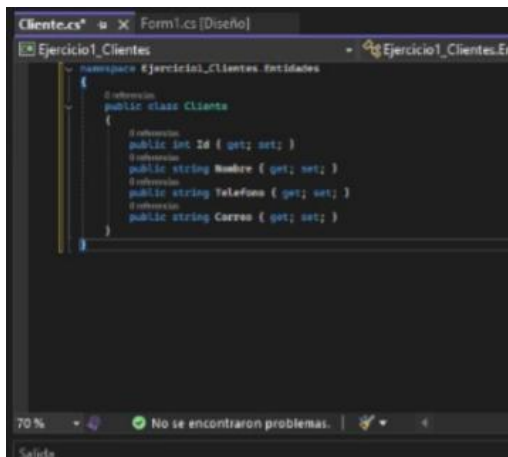
Configuración del formulario principal

se muestra el formulario principal del proyecto en el diseñador de Windows Forms. Se modificaron propiedades del formulario, incluyendo el nombre interno FrmClientes y el texto visible Registro de Clientes, para identificar correctamente la ventana principal del sistema.



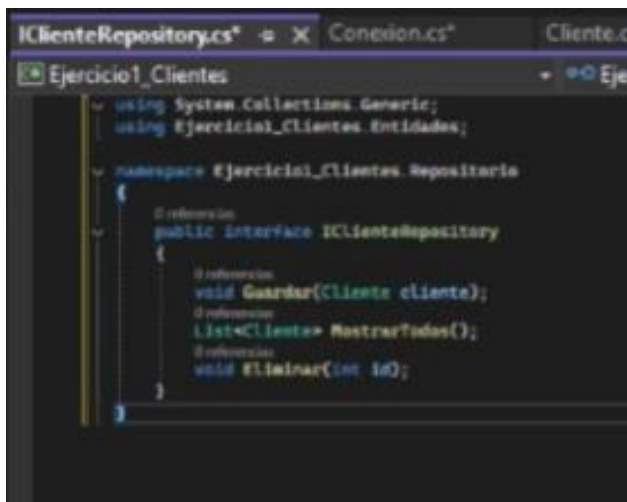
Diseño de la interfaz del formulario

se observa el diseño del formulario del sistema de clientes. Se agregaron los campos de texto para ingresar Nombre, Teléfono y Correo, además de los botones Guardar, Mostrar clientes y Eliminar seleccionado. También se incorporó un DataGridView para mostrar los clientes registrados.



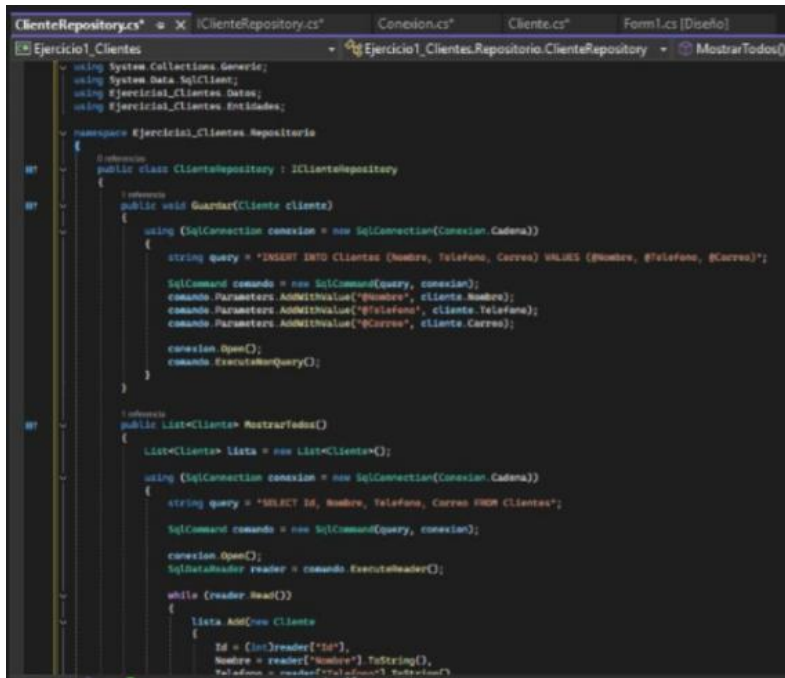
Creación de la entidad Cliente

se muestra la clase Cliente.cs, creada dentro de la carpeta Entidades. Esta clase representa la estructura de los datos del cliente, incluyendo las propiedades Id, Nombre, Teléfono y Correo, que corresponden a los campos de la tabla en la base de datos.



Creación de la interfaz del repositorio

se observa la interfaz IClienteRepository.cs, ubicada en la carpeta Repositorio. En ella se declararon los métodos principales que debe cumplir el sistema: Guardar, MostrarTodos y Eliminar, aplicando el patrón Repository solicitado en el ejercicio.



```
using System.Collections.Generic;
using System.Data.SqlClient;
using Ejercicio1_Clientes.Datos;
using Ejercicio1_Clientes.Entidades;

namespace Ejercicio1_Clientes.Repositorio
{
    public class ClienteRepository : IClienteRepository
    {
        public void Guardar(Cliente cliente)
        {
            using (SqlConnection conexion = new SqlConnection(Conexion.Cadena))
            {
                string query = "INSERT INTO Clientes (Nombre, Telefono, Correo) VALUES (@Nombre, @Telefono, @Correo)";

                SqlCommand comando = new SqlCommand(query, conexion);
                comando.Parameters.AddWithValue("@Nombre", cliente.Nombre);
                comando.Parameters.AddWithValue("@Telefono", cliente.Telefono);
                comando.Parameters.AddWithValue("@Correo", cliente.Correo);

                conexion.Open();
                comando.ExecuteNonQuery();
            }
        }

        public List<Cliente> MostrarTodos()
        {
            List<Cliente> lista = new List<Cliente>();

            using (SqlConnection conexion = new SqlConnection(Conexion.Cadena))
            {
                string query = "SELECT Id, Nombre, Telefono, Correo FROM Clientes";

                SqlCommand comando = new SqlCommand(query, conexion);

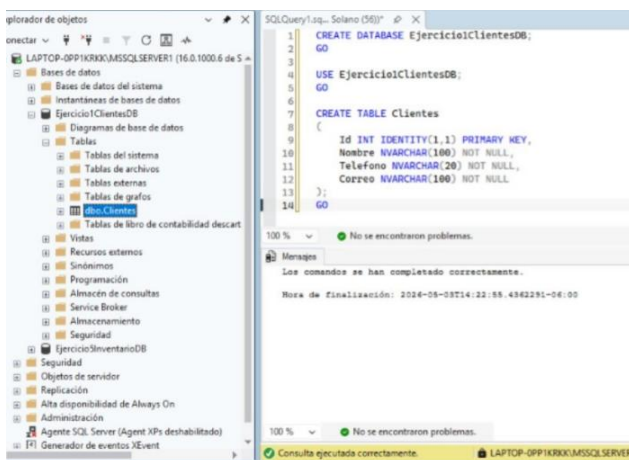
                conexion.Open();
                SqlDataReader reader = comando.ExecuteReader();

                while (reader.Read())
                {
                    lista.Add(new Cliente
                    {
                        Id = (int)reader["Id"],
                        Nombre = reader["Nombre"].ToString(),
                        Telefono = reader["Telefono"].ToString(),
                        Correo = reader["Correo"].ToString()
                    });
                }
            }

            return lista;
        }
    }
}
```

Programación del repositorio de clientes

se muestra parte del código de la clase ClienteRepository.cs. Esta clase implementa la interfaz del repositorio y contiene las operaciones necesarias para conectarse con SQL Server, guardar clientes, consultar los registros y eliminar clientes de la base de datos.



```
CREATE DATABASE Ejercicio1ClientesDB;
GO

USE Ejercicio1ClientesDB;
GO

CREATE TABLE Clientes
(
    Id INT IDENTITY(1,1) PRIMARY KEY,
    Nombre NVARCHAR(100) NOT NULL,
    Telefono NVARCHAR(20) NOT NULL,
    Correo NVARCHAR(100) NOT NULL
);
GO
```

Creación de la base de datos y tabla Clientes

se muestra SQL Server Management Studio con el script utilizado para crear la base de datos Ejercicio1ClientesDB y la tabla Clientes. También se observa que la tabla dbo.Clientes fue creada correctamente dentro de la base de datos.

```

Ejercicio1_Clientes
Ejercicio1_Clientes.FrmClientes

using System;
using System.Windows.Forms;
using Ejercicio1_Clientes.Entidades;
using Ejercicio1_Clientes.Repositorio;

namespace Ejercicio1_Clientes
{
    [reflection]
    public partial class FrmClientes : Form
    {
        private readonly IClienteRepository _repositorio;

        [reflection]
        public FrmClientes()
        {
            InitializeComponent();
            _repositorio = new ClienteRepository();
        }

        [reflection]
        private void FrmClientes_Load(object sender, EventArgs e)
        {
            // Esta método queda vacío.
            // Se agrega porque el diseñador lo está llamando.
        }

        [reflection]
        private void LimpiarCampos()
        {
            txtNombre.Clear();
            txtTelefono.Clear();
            txtCorreo.Clear();
            txtNombre.Focus();
        }

        [reflection]
        private bool ValidarCampos()
        {
            if (string.IsNullOrWhiteSpace(txtNombre.Text))
            {
                MessageBox.Show("Debe ingresar el nombre del cliente.");
                txtNombre.Focus();
                return false;
            }

            foreach (char letra in txtNombre.Text)
            {
                if (!char.IsLetter(letra) || !char.IsWhiteSpace(letra))
            }
        }
    }
}

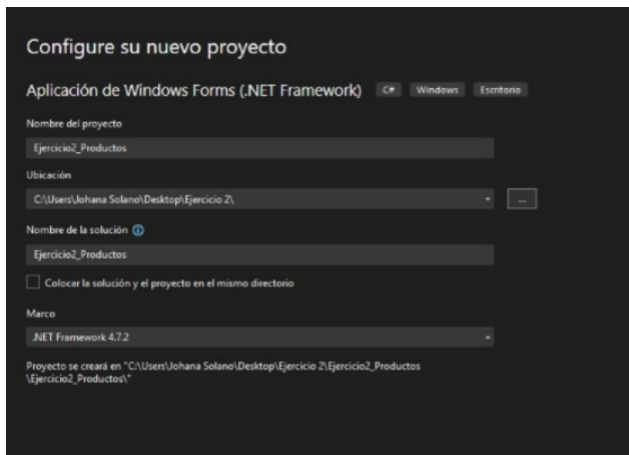
```

Programación del formulario y validaciones

se observa el código del formulario FrmClientes. En esta parte se programaron las funciones principales del formulario, incluyendo la validación de campos vacíos, validación de nombre con solo letras, teléfono con solo números y validación básica del correo electrónico.

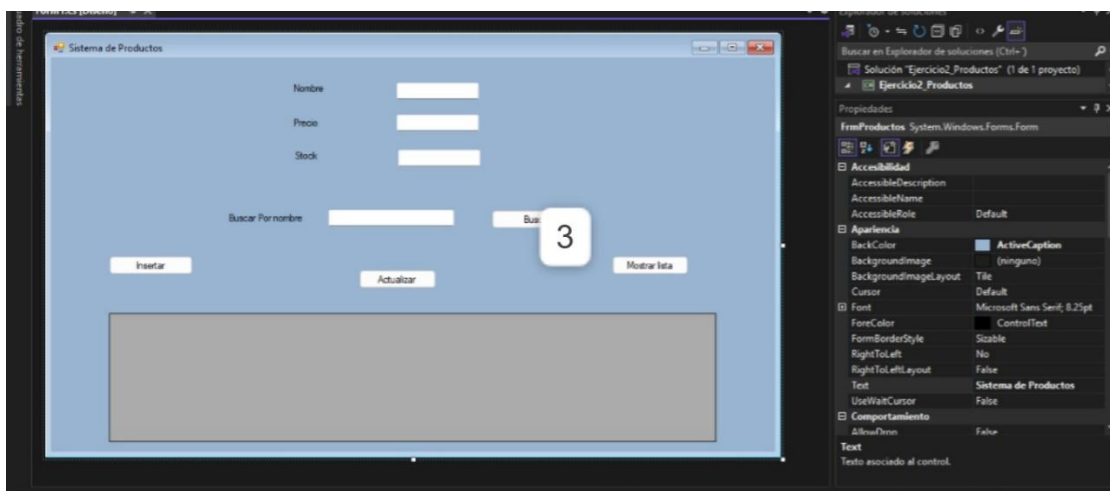
Conexión de eventos de los botones

se muestra la conexión del evento Click de los botones del formulario. Estos eventos permiten que los botones ejecuten sus respectivas funciones, como guardar clientes, mostrar los registros en el DataGridView y eliminar el cliente seleccionado.



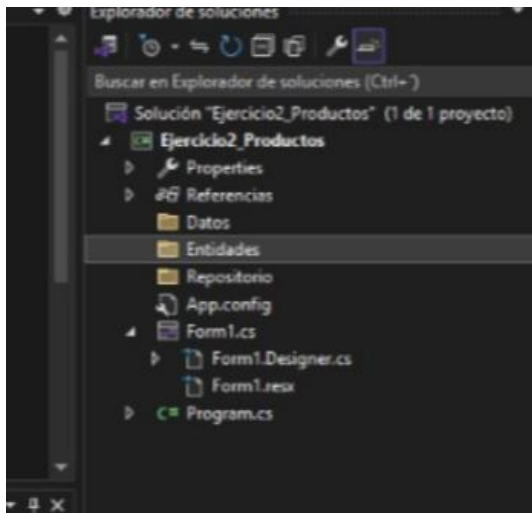
Configuración inicial del proyecto

observa la configuración del nuevo proyecto en Visual Studio. Se seleccionó una Aplicación de Windows Forms (.NET Framework), asignando como nombre del proyecto y de la solución Ejercicio2_Productos. También se seleccionó el framework .NET Framework 4.7.2.



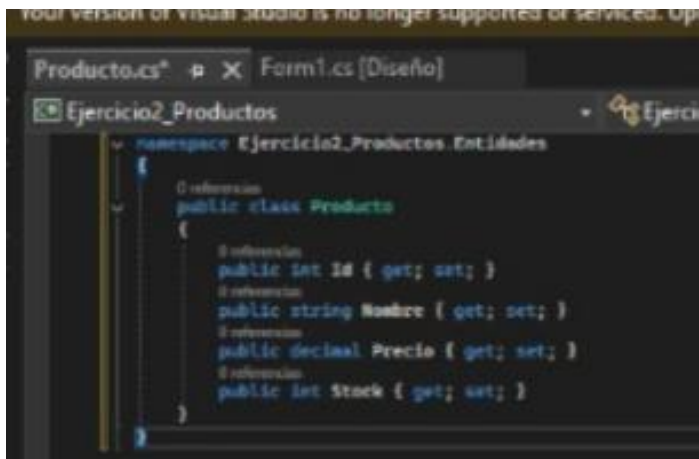
Diseño del formulario principal

se muestra el formulario principal con el título Sistema de Productos. Se agregaron los campos para ingresar Nombre, Precio y Stock, además del campo para buscar por nombre, los botones de acción y el DataGridView para mostrar la información.



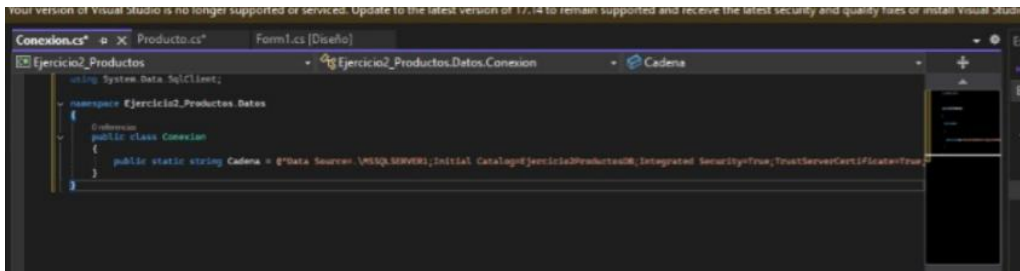
Creación de carpetas dentro del proyecto

se observa la organización del proyecto en el Explorador de soluciones. Se crearon las carpetas Datos, Entidades y Repositorio, con el objetivo de separar el código según su función y mantener una estructura ordenada.



Creación de la entidad Producto

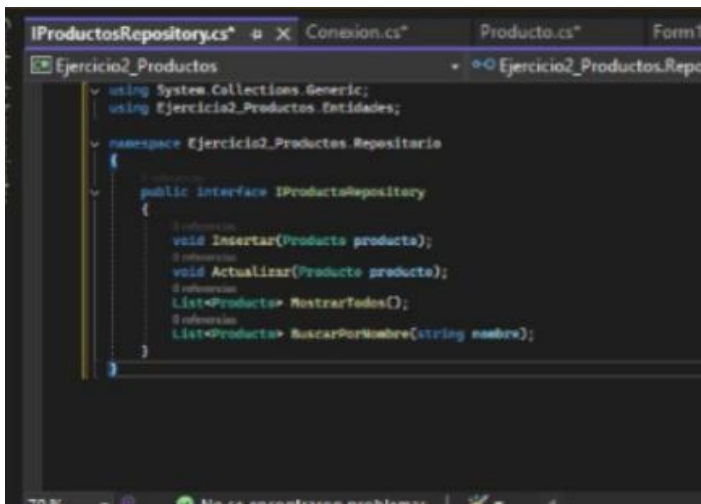
se muestra la clase Producto.cs, ubicada dentro de la carpeta Entidades. Esta clase representa la estructura de los datos del producto, incluyendo las propiedades Id, Nombre, Precio y Stock.



Creación

de la clase Conexión

muestra la clase Conexion.cs, ubicada dentro de la carpeta Datos. Esta clase contiene la cadena de conexión hacia SQL Server, permitiendo que el programa pueda comunicarse con la base de datos Ejercicio2ProductosDB.



Creación de la interfaz del repositorio

se observa la interfaz IProductoRepository.cs, ubicada en la carpeta Repositorio. En ella se declararon los métodos principales del sistema: Insertar, Actualizar, MostrarTodos y BuscarPorNombre.

```
 Ejercicio2_Productos
 Ejercicio2_Productos.Repositorio.ProductoReposit
 Mostrar Todos()

using System.Collections.Generic;
using System.Data.SqlClient;
using Ejercicio2_Productos.Datos;
using Ejercicio2_Productos.Entidades;

namespace Ejercicio2_Productos.Repositorio
{
    [reference]
    public class ProductoRepository : IProductoRepository
    {
        [reference]
        public void Insertar(Producto producto)
        {
            using (SqlConnection conexion = new SqlConnection(Conexion.Cadena))
            {
                string query = "INSERT INTO Productos (Nombre, Precio, Stock) VALUES (@Nombre, @Precio, @Stock)";

                SqlCommand comando = new SqlCommand(query, conexion);
                comando.Parameters.AddWithValue("@Nombre", producto.Nombre);
                comando.Parameters.AddWithValue("@Precio", producto.Precio);
                comando.Parameters.AddWithValue("@Stock", producto.Stock);

                conexion.Open();
                comando.ExecuteNonQuery();
            }
        }

        [reference]
        public void Actualizar(Producto producto)
        {
            using (SqlConnection conexion = new SqlConnection(Conexion.Cadena))
            {
                string query = "UPDATE Productos SET Nombre = @Nombre, Precio = @Precio, Stock = @Stock WHERE ID = @ID";

                SqlCommand comando = new SqlCommand(query, conexion);
                comando.Parameters.AddWithValue("@ID", producto.Id);
                comando.Parameters.AddWithValue("@Nombre", producto.Nombre);
                comando.Parameters.AddWithValue("@Precio", producto.Precio);
                comando.Parameters.AddWithValue("@Stock", producto.Stock);

                conexion.Open();
                comando.ExecuteNonQuery();
            }
        }
    }
}
```

Programación del repositorio de productos muestra parte del código de la clase `ProductoRepository.cs`. Esta clase implementa los métodos definidos en la interfaz y realiza las operaciones de insertar, actualizar, mostrar y buscar productos en la base de datos mediante ADO.NET.

```
 Ejercicio2_Productos
 Ejercicio2_Productos.Repositorio.ProductoReposit
 Mostrar Todos()

CREATE DATABASE Ejercicio2ProductosDB;
GO

USE Ejercicio2ProductosDB;
GO

CREATE TABLE Productos
(
    Id INT IDENTITY(1,1) PRIMARY KEY,
    Nombre NVARCHAR(100) NOT NULL,
    Precio DECIMAL(18,2) NOT NULL,
    Stock INT NOT NULL
);
GO
```

Creación de la base de datos y tabla Productos

SQL Server Management Studio con el script utilizado para crear la base de datos `Ejercicio2ProductosDB` y la tabla `Productos`. La tabla contiene los campos `Id`, `Nombre`, `Precio` y `Stock`.

```
Form1.cs*  X ProductoRepository.cs  IProductoRepository.cs*  Conexion.cs*  Producto.cs*  Form1.cs*
+ Ejercicio2_Productos.FrmProductos
+ Ejercicio2_Productos.Repository
+ Ejercicio2_Productos.Repository

namespace Ejercicio2_Productos
{
    public partial class FrmProductos : Form
    {
        private readonly IProductoRepository _repository;

        public FrmProductos()
        {
            InitializeComponent();
            _repository = new ProductoRepository();
        }

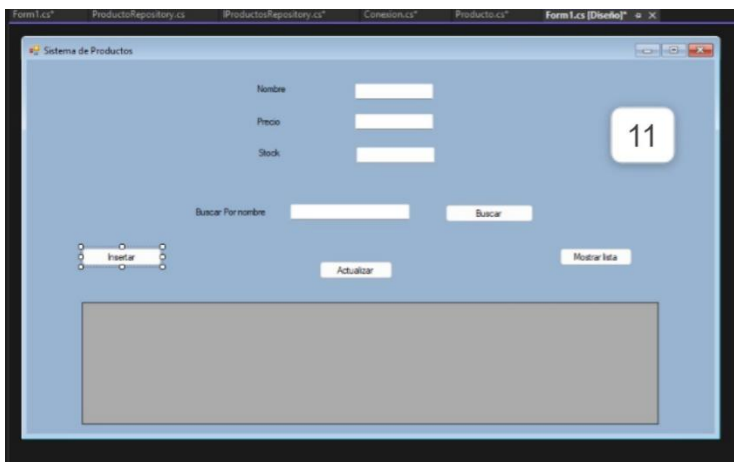
        private void FrmProductos_Load(object sender, EventArgs e)
        {
            // Este método queda vacío.
            // Se agrega porque el diseñador lo puede estar llamando.
        }

        private void LimpiarCampos()
        {
            txtNombre.Clear();
            txtPrecio.Clear();
            txtStock.Clear();
            txtBuscar.Clear();
            txtInsertar.Focus();
        }

        private void ValidarCampos()
        {
            if (string.IsNullOrEmpty(txtNombre.Text))
            {
                MessageBox.Show("Debe ingresar el nombre del producto.");
                txtNombre.Focus();
            }
            return false;
        }
    }
}
```

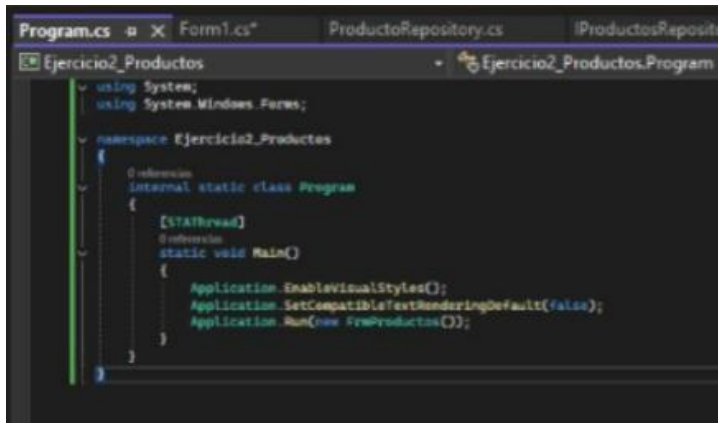
Programación del formulario y validaciones

observa el código del formulario FrmProductos. En este archivo se programaron las funciones principales del sistema, incluyendo la inserción, actualización, búsqueda, visualización de productos y validación de los datos ingresados.



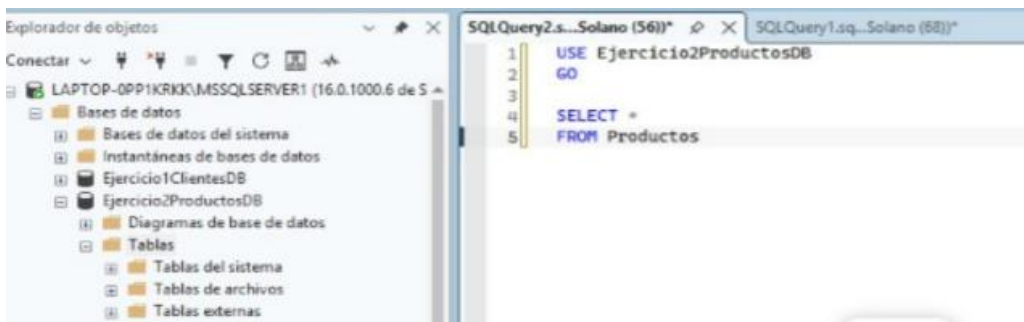
Conexión de eventos de los botones

se muestra la conexión del evento Click del botón Insertar. Esta configuración permite que el botón ejecute el método correspondiente al momento de ser presionado. Este mismo procedimiento se realizó con los demás botones del formulario.



Configuración del archivo Program.cs

el archivo Program.cs del proyecto. En este archivo se configuró el inicio de la aplicación para que ejecute el formulario principal FrmProductos mediante la instrucción `Application.Run(new FrmProductos());`.



Verificación de productos en SQL Server

consulta realizada en SQL Server Management Studio para verificar los productos guardados en la tabla Productos. Se observa que los registros insertados desde el programa aparecen correctamente en la base de datos, confirmando que la conexión y las operaciones funcionan.

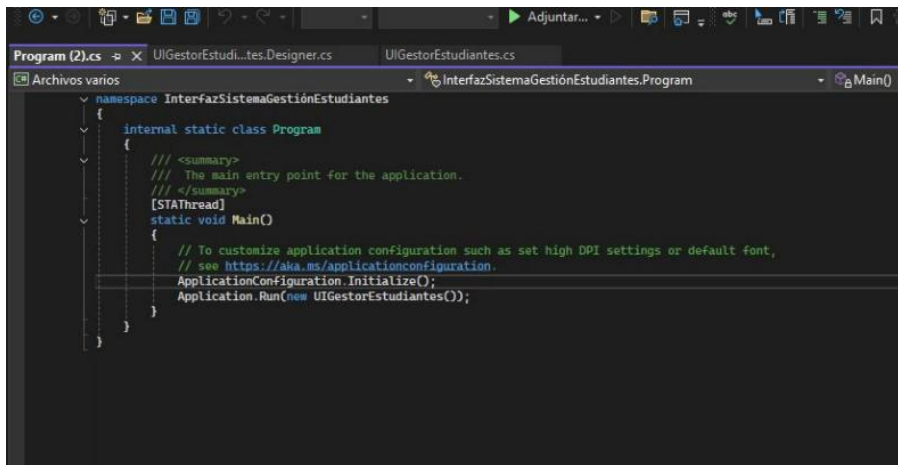
////////////////////////////////////



////////////////////////////////////

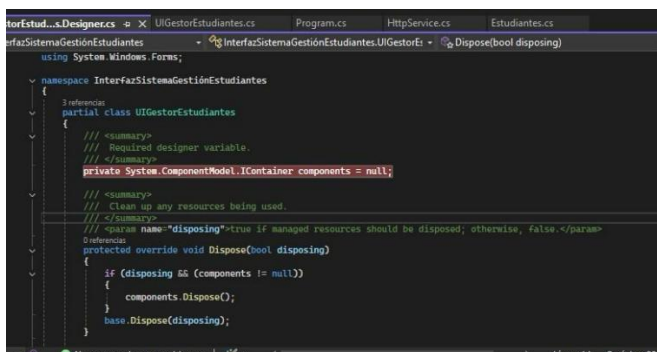


////////////////////////////////////



```
namespace InterfazSistemaGestiónEstudiantes
{
    internal static class Program
    {
        /// <summary>
        /// The main entry point for the application.
        /// </summary>
        [STAThread]
        static void Main()
        {
            // To customize application configuration such as set high DPI settings or default font,
            // see https://aka.ms/applicationconfiguration.
            ApplicationConfiguration.Initialize();
            Application.Run(new UIGestorEstudiantes());
        }
    }
}
```

aparece el archivo Program.cs, que es el punto de inicio del programa y utiliza Application.Run() para ejecutar el formulario principal.

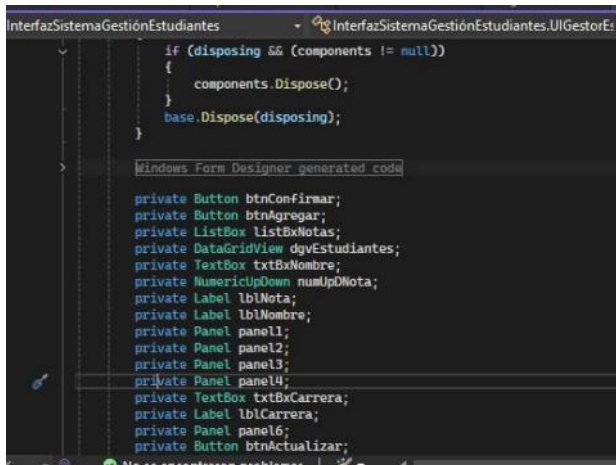


```
using System.Windows.Forms;

namespace InterfazSistemaGestiónEstudiantes
{
    partial class UIGestorEstudiantes
    {
        /// <summary>
        /// Required designer variable.
        /// </summary>
        private System.ComponentModel.IContainer components = null;

        /// <summary>
        /// Clean up any resources being used.
        /// </summary>
        /// <param name="disposing">true if managed resources should be disposed; otherwise, false.</param>
        protected override void Dispose(bool disposing)
        {
            if (disposing && (components != null))
            {
                components.Dispose();
            }
            base.Dispose(disposing);
        }
    }
}
```

se refuerza el diseño del formulario, mostrando botones de acciones y campos de entrada que permiten al usuario interactuar con el sistema.



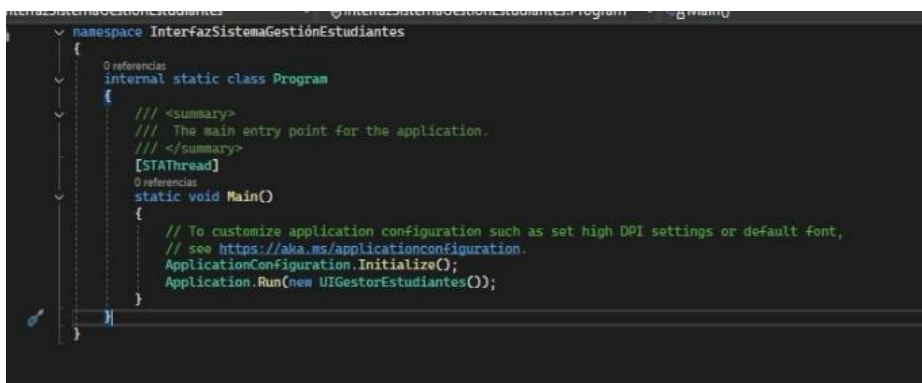
```
InterfazSistemaGestiónEstudiantes - InterfazSistemaGestiónEstudiantes.UIGestorEstudiantes

if (disposing && (components != null))
{
    components.Dispose();
}
base.Dispose(disposing);

Windows Form Designer generated code

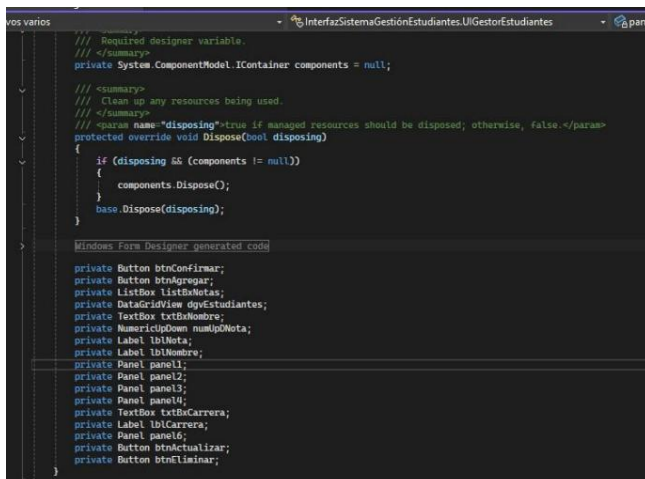
private Button btnConfirmar;
private Button btnAgregar;
private ListBox listBxNotas;
private DataGridView dgvEstudiantes;
private TextBox txtBxNombre;
private NumericUpDown numUpdNota;
private Label lblNota;
private Label lblNombre;
private Panel panel1;
private Panel panel2;
private Panel panel3;
private Panel panel4;
private TextBox txtBxCarrera;
private Label lblCarrera;
private Panel panel6;
private Button btnActualizar;
```

se presenta el controlador de la API, encargado de manejar las solicitudes desde la interfaz mediante rutas y atributos como [HttpGet].



```
namespace InterfazSistemaGestiónEstudiantes
{
    0 referencias
    internal static class Program
    {
        /// <summary>
        /// The main entry point for the application.
        /// </summary>
        [STAThread]
        0 referencias
        static void Main()
        {
            // To customize application configuration such as set high DPI settings or default font,
            // see https://aka.ms/applicationconfiguration.
            ApplicationConfiguration.Initialize();
            Application.Run(new UIGestorEstudiantes());
        }
    }
}
```

se muestra el método que obtiene todos los estudiantes desde la base de datos usando programación asíncrona para mejorar el rendimiento.



```
vos varios - InterfazSistemaGestiónEstudiantes.UIGestorEstudiantes

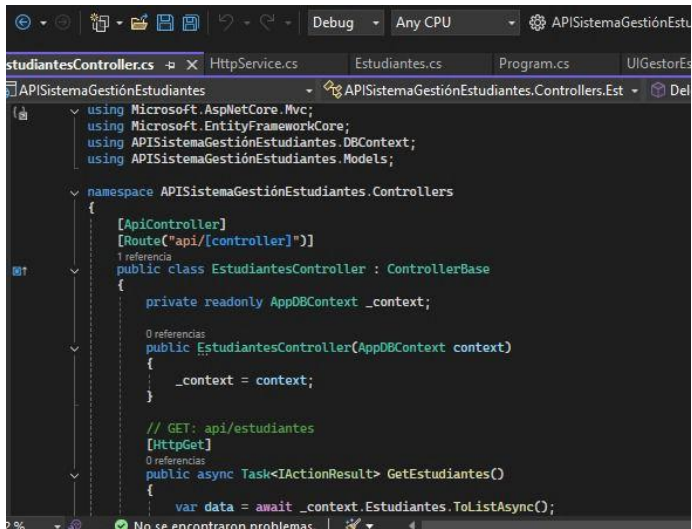
/// Required designer variable.
/// <summary>
private System.ComponentModel.IContainer components = null;

/// <summary>
/// Clean up any resources being used.
/// </summary>
/// <param name="disposing">true if managed resources should be disposed; otherwise, false.</param>
protected override void Dispose(bool disposing)
{
    if (disposing && (components != null))
    {
        components.Dispose();
    }
    base.Dispose(disposing);
}

Windows Form Designer generated code

private Button btnConfirmar;
private Button btnAgregar;
private ListBox listBxNotas;
private DataGridView dgvEstudiantes;
private TextBox txtBxNombre;
private NumericUpDown numUpdNota;
private Label lblNota;
private Label lblNombre;
private Panel panel1;
private Panel panel2;
private Panel panel3;
private Panel panel4;
private Panel panel6;
private TextBox txtBxCarrera;
private Label lblCarrera;
private Panel panel6;
private Button btnActualizar;
private Button btnEliminar;
```

En la séptima imagen se observa el método que obtiene un estudiante por su ID, permitiendo consultar registros específicos.



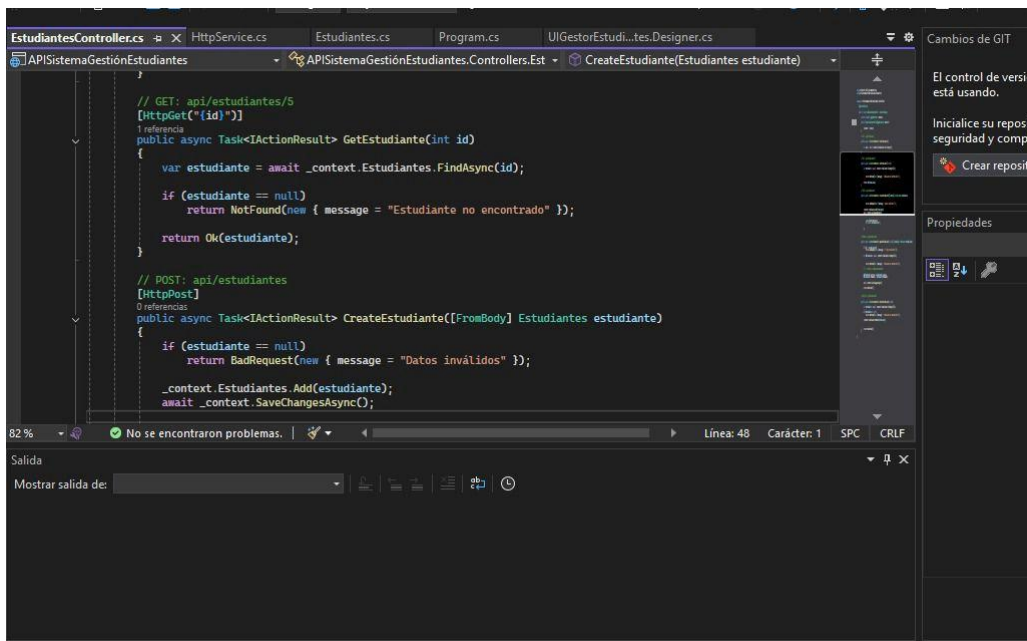
```
using Microsoft.AspNetCore.Mvc;
using Microsoft.EntityFrameworkCore;
using APISistemaGestiónEstudiantes.DBContext;
using APISistemaGestiónEstudiantes.Models;

namespace APISistemaGestiónEstudiantes.Controllers
{
    [ApiController]
    [Route("api/[controller]")]
    public class EstudiantesController : ControllerBase
    {
        private readonly AppDbContext _context;

        public EstudiantesController(AppDbContext context)
        {
            _context = context;
        }

        // GET: api/estudiantes
        [HttpGet]
        public async Task<IActionResult> GetEstudiantes()
        {
            var data = await _context.Estudiantes.ToListAsync();
        }
    }
}
```

En la octava imagen se presenta el método POST, que permite insertar nuevos estudiantes en la base de datos.



```
// GET: api/estudiantes/5
[HttpGet("{id}")]
public async Task<IActionResult> GetEstudiante(int id)
{
    var estudiante = await _context.Estudiantes.FindAsync(id);

    if (estudiante == null)
        return NotFound(new { message = "Estudiante no encontrado" });

    return Ok(estudiante);
}

// POST: api/estudiantes
[HttpPost]
public async Task<IActionResult> CreateEstudiante([FromBody] Estudiantes estudiante)
{
    if (estudiante == null)
        return BadRequest(new { message = "Datos inválidos" });

    _context.Estudiantes.Add(estudiante);
    await _context.SaveChangesAsync();
}
```

En la novena imagen se muestra el método PUT, utilizado para actualizar la información de un estudiante existente.

```

// UPDATE: api/estudiantes/5
[HttpPut("{id}")]
0 referencias
public async Task<ActionResult> UpdateEstudiante(int id, [FromBody] Estudiantes estudiante)
{
    if (id != estudiante.Id)
        return BadRequest(new { message = "El ID no coincide" });

    var dbEstudiante = await _context.Estudiantes.FindAsync(id);

    if (dbEstudiante == null)
        return NotFound(new { message = "Estudiante no encontrado" });

    // * actualizar campos manualmente
    dbEstudiante.Nombre = estudiante.Nombre;
    dbEstudiante.Carrera = estudiante.Carrera;
    dbEstudiante.Promedio = estudiante.Promedio;

    await _context.SaveChangesAsync();

    return NoContent();
}

```

se observa el método DELETE, que elimina un estudiante de la base de datos.

```

return NoContent();
}

// DELETE: api/estudiantes/5
[HttpDelete("{id}")]
0 referencias
public async Task<ActionResult> DeleteEstudiante(int id)
{
    var estudiante = await _context.Estudiantes.FindAsync(id);

    if (estudiante == null)
        return NotFound(new { message = "Estudiante no encontrado" });

    _context.Estudiantes.Remove(estudiante);
    await _context.SaveChangesAsync();

    return NoContent();
}
}

```

se introduce la clase HttpServices, que conecta la interfaz con la API usando HttpClient.

```

using System.Text;
using System.Text.Json;
using InterfazSistemaGestiónEstudiantes.Models;

namespace InterfazSistemaGestiónEstudiantes.Services
{
    3 referencias
    public class HttpService
    {
        private readonly HttpClient _httpClient;

        private const string BaseUrl = "http://localhost:5144/api/estudiantes";

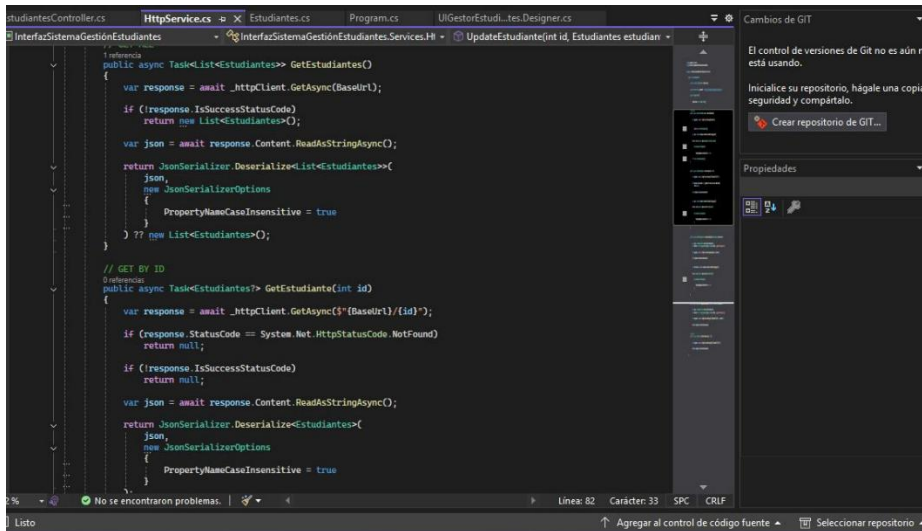
        1 referencia
        public HttpService()
        {
            _httpClient = new HttpClient();
        }

        // GET ALL
        1 referencia
        public async Task<List<Estudiantes>> GetEstudiantes()
        {
            var response = await _httpClient.GetAsync(BaseUrl);

            if (!response.IsSuccessStatusCode)

```

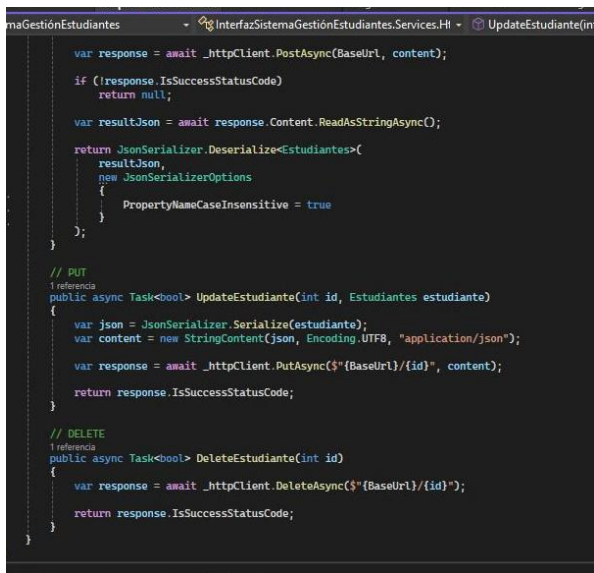
se muestra el método GET del cliente, que obtiene los estudiantes y convierte los datos JSON a objetos de C#.



```
public async Task<List<Estudiantes>> GetEstudiantes()
{
    var response = await _httpClient.GetAsync(BaseUrl);
    if (!response.IsSuccessStatusCode)
        return new List<Estudiante>();
    var json = await response.Content.ReadAsStringAsync();
    return JsonSerializer.Deserialize<List<Estudiantes>>(
        json,
        new JsonSerializerOptions
        {
            PropertyNameCaseInsensitive = true
        }
    ) ?? new List<Estudiantes>();
}

// GET BY ID
1 referencia
public async Task<Estudiantes?> GetEstudiante(int id)
{
    var response = await _httpClient.GetAsync($"{BaseUrl}/{id}");
    if (response.StatusCode == System.Net.HttpStatusCode.NotFound)
        return null;
    if (!response.IsSuccessStatusCode)
        return null;
    var json = await response.Content.ReadAsStringAsync();
    return JsonSerializer.Deserialize<Estudiantes>(
        json,
        new JsonSerializerOptions
        {
            PropertyNameCaseInsensitive = true
        }
    );
}
```

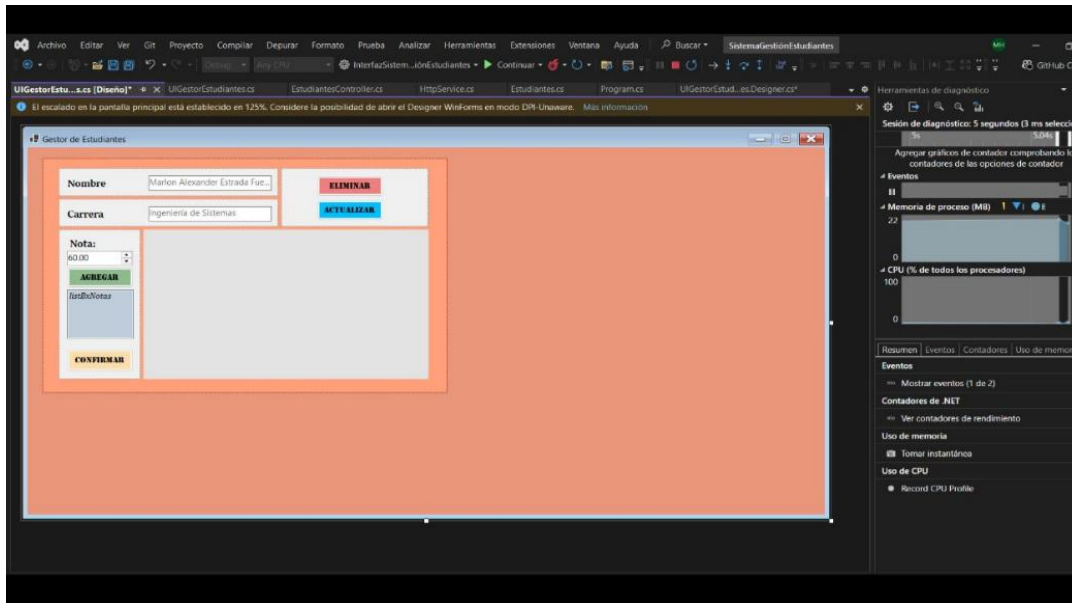
En la decimotercera imagen se presenta el método para obtener un estudiante por ID desde la API, validando la respuesta recibida.



```
// PUT
1 referencia
public async Task<bool> UpdateEstudiante(int id, Estudiantes estudiante)
{
    var json = JsonSerializer.Serialize(estudiante);
    var content = new StringContent(json, Encoding.UTF8, "application/json");
    var response = await _httpClient.PutAsync($"{BaseUrl}/{id}", content);
    return response.IsSuccessStatusCode;
}

// DELETE
1 referencia
public async Task<bool> DeleteEstudiante(int id)
{
    var response = await _httpClient.DeleteAsync($"{BaseUrl}/{id}");
    return response.IsSuccessStatusCode;
}
```

se observan los métodos POST, PUT y DELETE del cliente, que permiten completar las operaciones CRUD desde la interfaz.



se muestra la interfaz en ejecución, donde el usuario puede ingresar datos y verlos en una tabla.

```

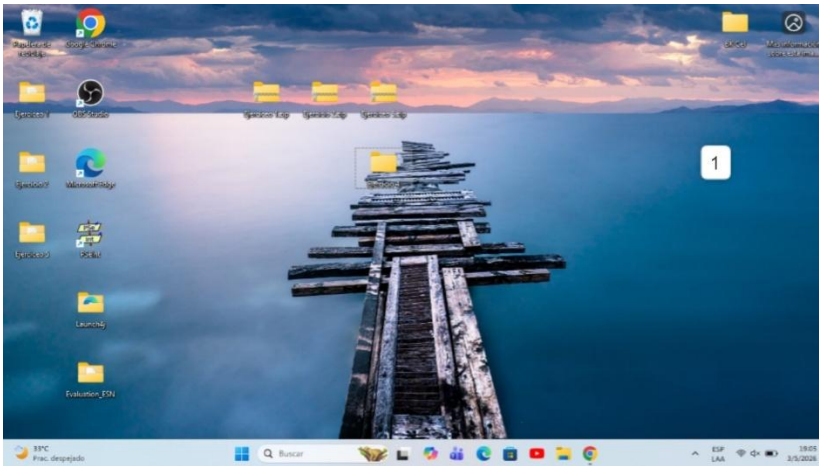
DB-SistemaE...es (sa (80))
1  -- CREAM BASE SI NO EXISTE
2  IF NOT EXISTS (SELECT * FROM sys.databases WHERE name = 'BDSistemaEstudiantes')
3  BEGIN
4      CREATE DATABASE BDSistemaEstudiantes;
5  END
6  GO
7
8  USE BDSistemaEstudiantes;
9  GO
10
11  CREATE TABLE Estudiantes(
12      Id INT IDENTITY(1,1) PRIMARY KEY,
13      Nombre NVARCHAR(150) NOT NULL,
14      Carrera NVARCHAR(150) NOT NULL,
15      Promedio DECIMAL(5,2) NOT NULL
16  );

```

se observa el script SQL, donde se crea la base de datos y la tabla Estudiantes con sus respectivos campos.

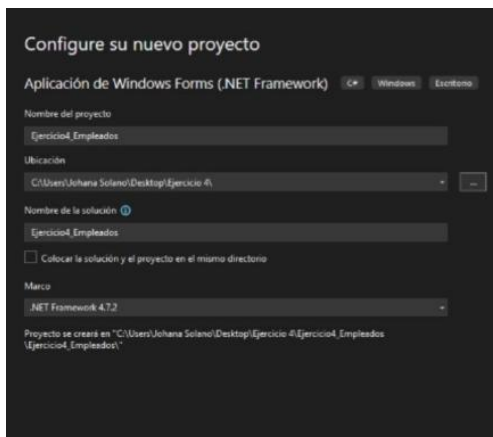
////////////////////////////////////

4) Ejercicio



Creación de la carpeta del ejercicio

la carpeta principal llamada Ejercicio 4, creada en el escritorio. Esta carpeta se utilizó para organizar todos los archivos correspondientes al desarrollo del sistema de empleados.



Configuración inicial del proyecto

En esta imagen se observa la configuración del nuevo proyecto en Visual Studio. Se seleccionó una Aplicación de Windows Forms (.NET Framework), asignando como nombre del proyecto y de la solución Ejercicio4_Empleados. También se seleccionó el framework .NET Framework 4.7.2, que será utilizado para desarrollar la aplicación.

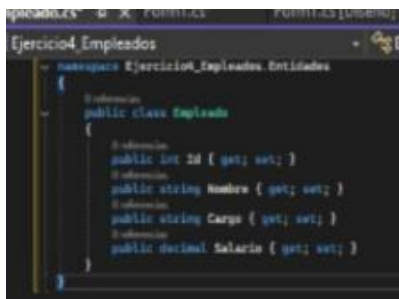
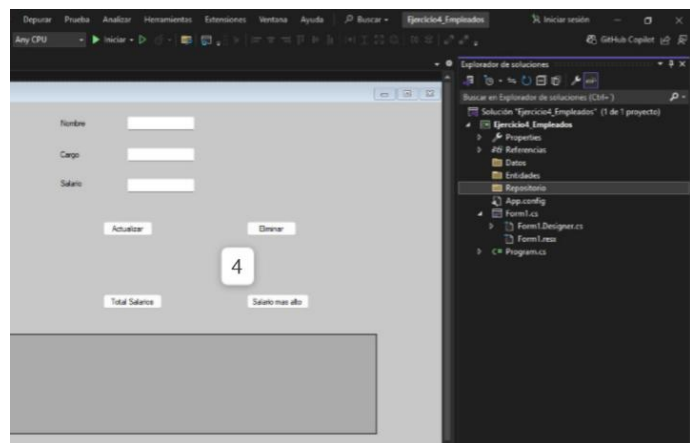


Diseño del formulario principal

En esta captura se muestra el formulario principal del sistema con el título Sistema de Empleados. Se agregaron los campos para ingresar Nombre, Cargo y Salario, además de los botones para guardar, actualizar, eliminar, mostrar empleados, calcular total de salarios y mostrar el salario más alto. También se incorporó un DataGridView para visualizar los empleados registrados.

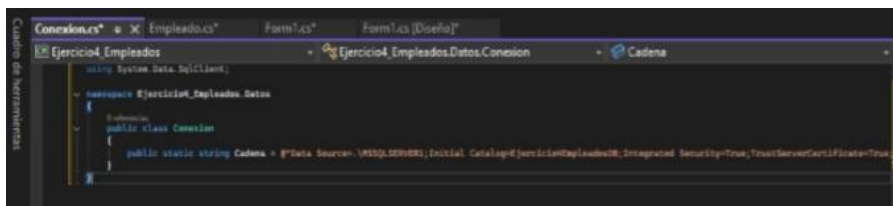
Creación de carpetas dentro del proyecto

En esta imagen se observa la organización del proyecto en el Explorador de soluciones. Se crearon las carpetas Datos, Entidades y Repositorio, con el objetivo de separar el código según su función y mantener una estructura ordenada dentro del proyecto.



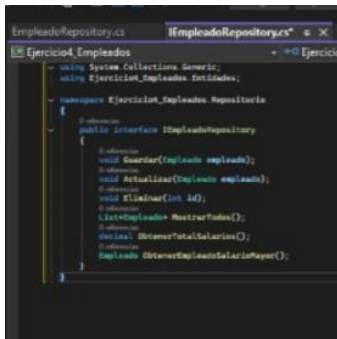
Creación de la entidad Empleado

En esta captura se muestra la clase Empleado.cs, ubicada dentro de la carpeta Entidades. Esta clase representa la estructura de los datos del empleado, incluyendo las propiedades Id, Nombre, Cargo y Salario, que corresponden a los campos de la tabla en la base de datos.



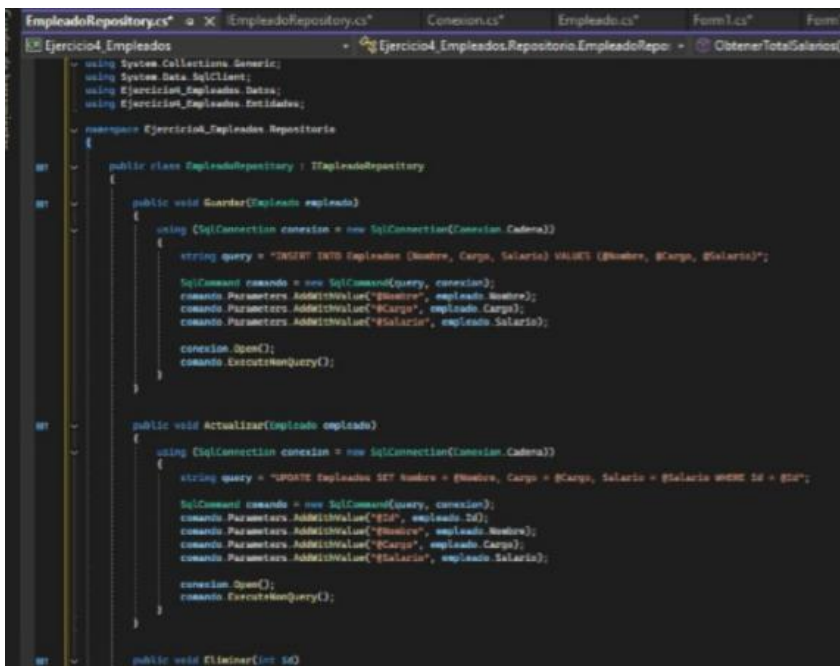
Creación de la clase Conexión

la clase Conexion.cs, ubicada dentro de la carpeta Datos. Esta clase contiene la cadena de conexión hacia SQL Server, permitiendo que la aplicación pueda comunicarse con la base de datos



Creación de la interfaz del repositorio

observa la interfaz IEmpleadoRepository.cs, ubicada en la carpeta Repositorio. En ella se declararon los métodos principales del sistema, como guardar, actualizar, eliminar, mostrar empleados, obtener el total de salarios y obtener el empleado con salario más alto, aplicando el patrón Repository.



Programación del repositorio de empleados

En esta imagen se muestra parte del código de la clase EmpleadoRepository.cs. Esta clase implementa los métodos definidos en la interfaz y realiza las operaciones de insertar, actualizar, eliminar, consultar empleados, calcular el total de salarios y obtener el empleado con mayor salario mediante ADO.NET.

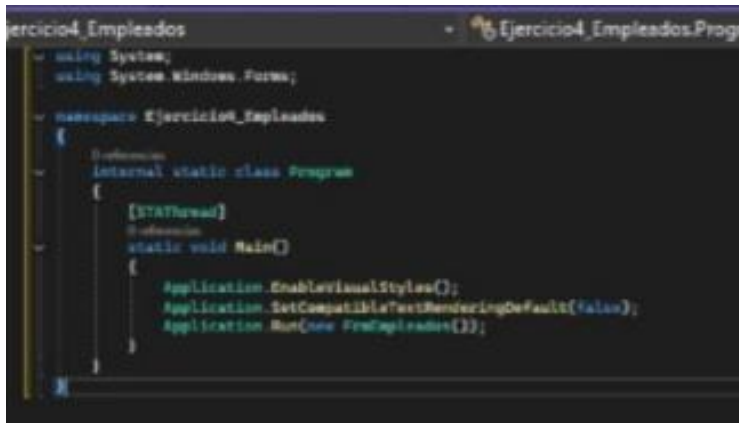
```
SQL Query 1.s... Solano (54)
1 CREATE DATABASE Ejercicio4EmpleadosDB;
2 GO
3
4 USE Ejercicio4EmpleadosDB;
5 GO
6
7 CREATE TABLE Empleados
8 (
9     Id INT IDENTITY(1,1) PRIMARY KEY,
10    Nombre NVARCHAR(100) NOT NULL,
11    Cargo NVARCHAR(100) NOT NULL,
12    Salario DECIMAL(18,2) NOT NULL
13 );
14 GO
```

Creación de la base de datos y tabla Empleados SQL Server Management Studio con el script utilizado para crear la base de datos Ejercicio4EmpleadosDB y la tabla Empleados. La tabla contiene los campos Id, Nombre, Cargo y Salario. También se observa que la consulta fue ejecutada correctamente.

[illegible]

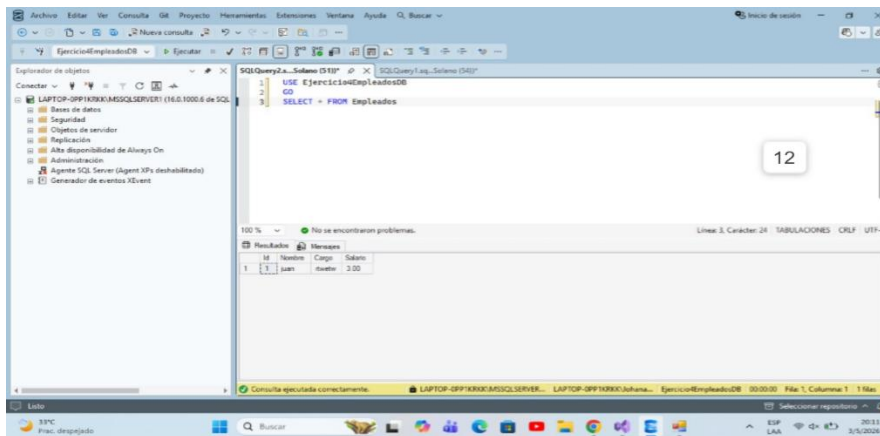
Programación del archivo Designer del formulario

En esta imagen se observa el archivo Form1.Designer.cs, donde se encuentra el código generado para los controles del formulario. En este archivo se configuran los labels, textBox, botones y el DataGridView, además de la conexión de los eventos necesarios para que los botones ejecuten sus funciones.



Configuración del archivo Program.cs

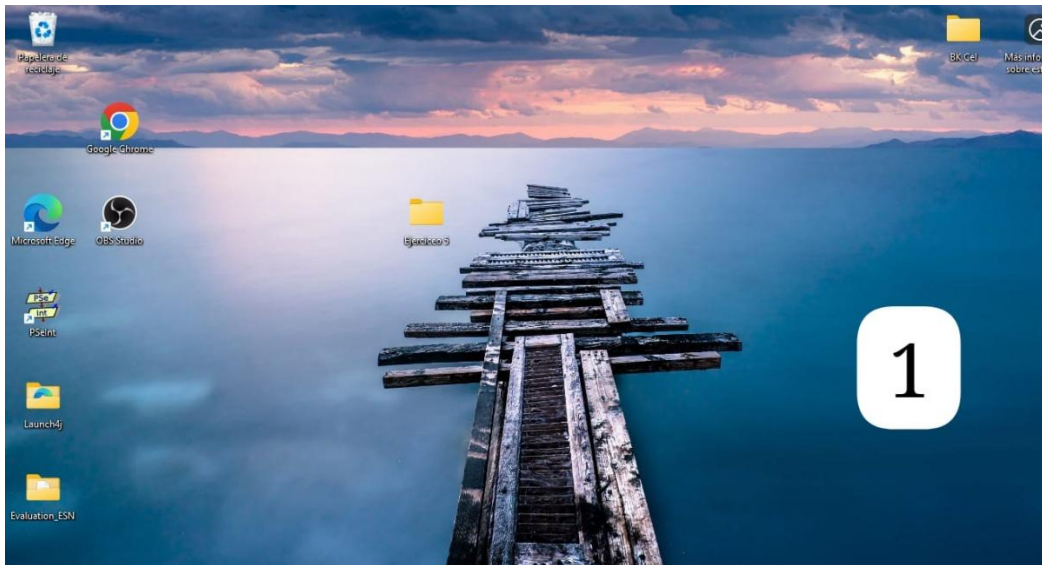
el archivo Program.cs del proyecto. En este archivo se configuró el inicio de la aplicación para que ejecute el formulario principal FrmEmpleados mediante la instrucción `Application.Run(new FrmEmpleados());`.



Verificación de empleados en SQL Server

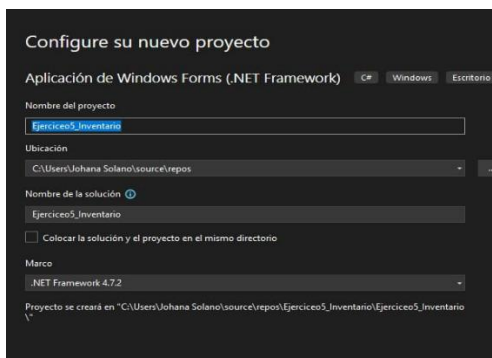
En esta imagen se muestra una consulta realizada en SQL Server Management Studio para verificar los registros guardados en la tabla Empleados. Se observa que los datos insertados desde el programa aparecen correctamente en la base de datos, confirmando que la conexión y las operaciones funcionan.

5) Ejercicio



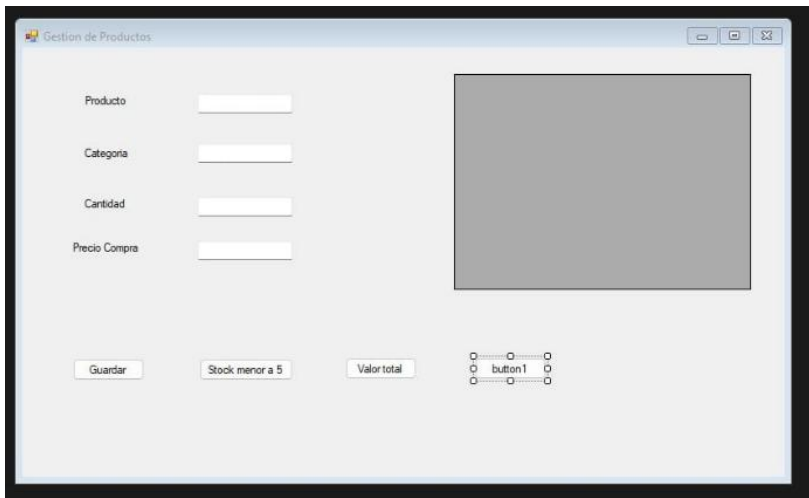
Creación de carpeta del ejercicio

se muestra la carpeta principal llamada Ejercicio 5, creada en el escritorio. Esta carpeta se utilizó para organizar todos los archivos relacionados con el desarrollo del ejercicio, manteniendo separado el proyecto correspondiente al control de inventario.



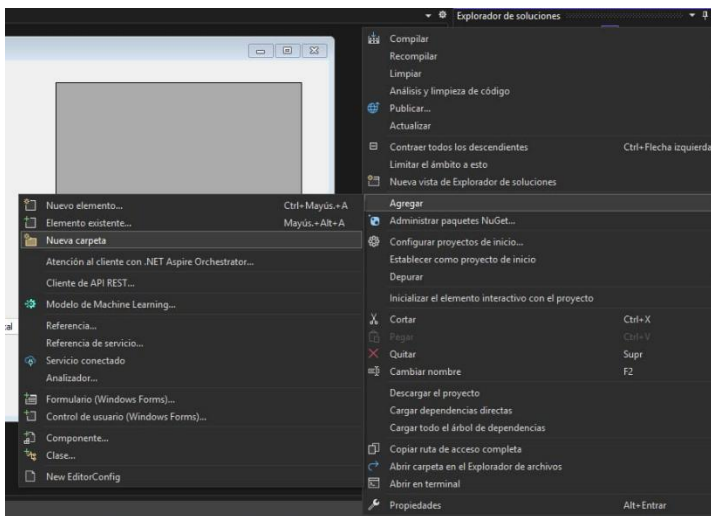
Configuración inicial del proyecto

En esta captura se observa la configuración del nuevo proyecto en Visual Studio. Se seleccionó una Aplicación de Windows Forms (.NET Framework), asignando como nombre del proyecto y de la solución Ejercicio5_Inventario. También se dejó seleccionado el framework .NET Framework 4.7.2, que será utilizado para desarrollar el formulario y conectarlo posteriormente con SQL Server.



Configuración del botón Mostrar todo

En esta captura se observa la configuración de uno de los botones del formulario. Se asignó el nombre interno btnMostrarTodo y el texto visible Mostrar todo. Este botón servirá para cargar en el DataGridView todos los productos registrados en la base de datos.



Creación de carpetas dentro del proyecto

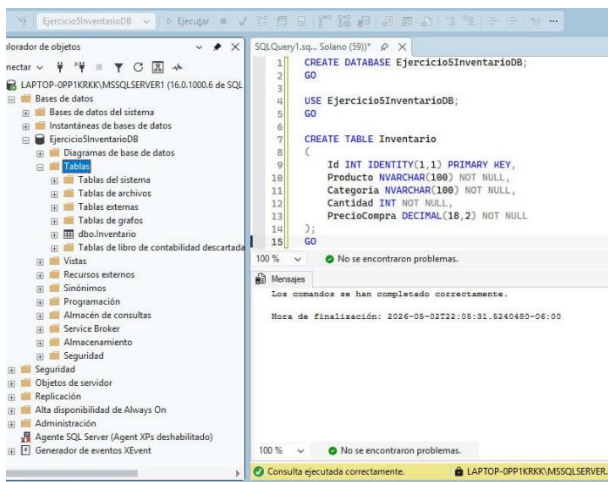
proceso de creación de carpetas dentro del proyecto desde el Explorador de soluciones. Esta organización permite separar mejor el código según su función, creando carpetas para entidades, datos y repositorio, lo cual facilita el mantenimiento del programa.



6

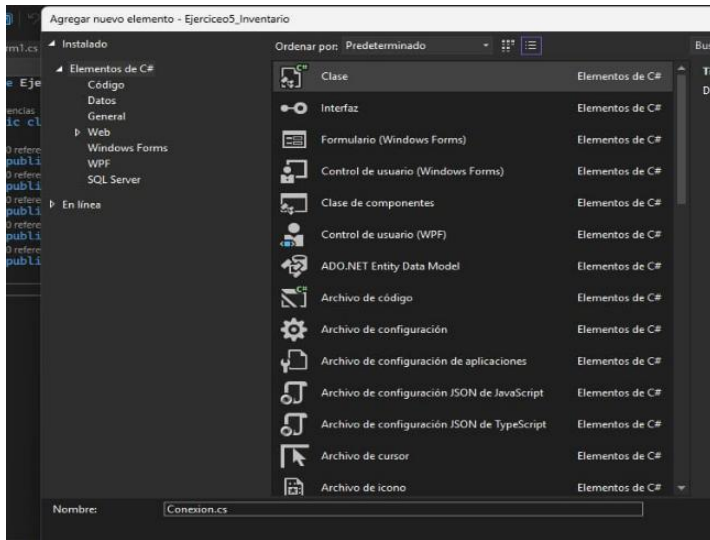
Diseño del formulario y estructura del proyecto

se muestra el formulario ya diseñado con sus controles principales: campos de texto para producto, categoría, cantidad y precio de compra; botones para guardar, mostrar stock bajo, calcular valor total y mostrar todos los registros; además del DataGridView. También se observa en el Explorador de soluciones la estructura organizada del proyecto.



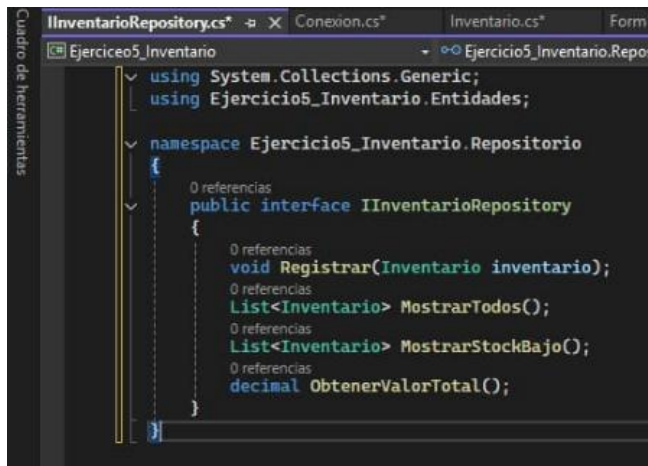
Creación de la base de datos y tabla Inventario

el script SQL utilizado para crear la base de datos Ejercicio5InventarioDB y la tabla Inventario. La tabla contiene los campos Id, Producto, Categoría, Cantidad y PrecioCompra. También se observa que la consulta fue ejecutada correctamente y que la tabla dbo. Inventario aparece en el Explorador de objetos.



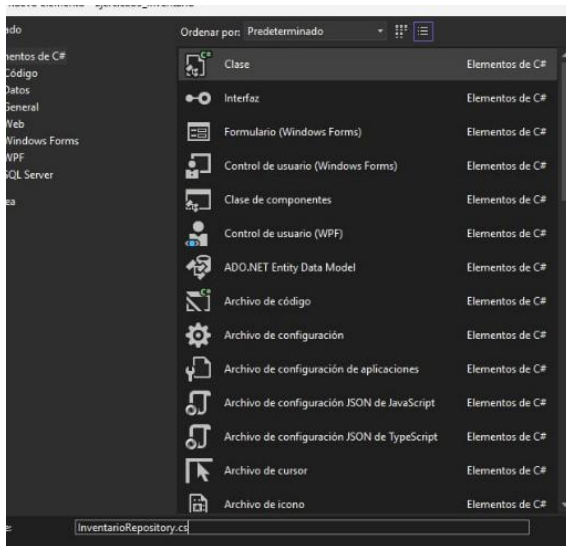
Creación de la clase Conexión

creación del archivo Conexion.cs dentro del proyecto. Esta clase se utilizará para guardar la cadena de conexión con SQL Server, permitiendo que el programa pueda comunicarse con la base de datos creada anteriormente.



Creación de la interfaz del repositorio:

el archivo IInventarioRepository.cs, donde se definió la interfaz del repositorio. En ella se declararon los métodos principales del sistema: registrar productos, mostrar todos los productos, mostrar productos con stock bajo y obtener el valor total del inventario.



Creación de la clase InventarioRepository

En esta captura se muestra el proceso de creación del archivo InventarioRepository.cs dentro de la carpeta Repositorio. Esta clase será la encargada de implementar los métodos definidos en la interfaz y realizar las operaciones directamente contra la base de datos.

```

using System.Collections.Generic;
using System.Data.SqlClient;
using Ejercicio5_Inventory.Datos;
using Ejercicio5_Inventory.Entidades;

namespace Ejercicio5_Inventory.Repositorio
{
    [Serializable]
    public class InventarioRepository : IInventarioRepository
    {
        [Serializable]
        public void Registrar(Inventario inventario)
        {
            using (SqlConnection conexion = new SqlConnection(Conexion.Cadena))
            {
                string query = "INSERT INTO Inventario (Producto, Categoria, Cantidad, PrecioCompra) VALUES (@Producto, @Categoria, @Cantidad, @PrecioCompra)";

                SqlCommand comando = new SqlCommand(query, conexion);
                comando.Parameters.AddWithValue("@Producto", inventario.Producto);
                comando.Parameters.AddWithValue("@Categoria", inventario.Categoria);
                comando.Parameters.AddWithValue("@Cantidad", inventario.Cantidad);
                comando.Parameters.AddWithValue("@PrecioCompra", inventario.PrecioCompra);

                conexion.Open();
                comando.ExecuteNonQuery();
            }
        }

        [Serializable]
        public List<Inventario> MostrarTodos()
        {
            List<Inventario> lista = new List<Inventario>();

            using (SqlConnection conexion = new SqlConnection(Conexion.Cadena))
            {
                string query = "SELECT Id, Producto, Categoria, Cantidad, PrecioCompra FROM Inventario";

                SqlCommand comando = new SqlCommand(query, conexion);

                conexion.Open();
                SqlDataReader reader = comando.ExecuteReader();

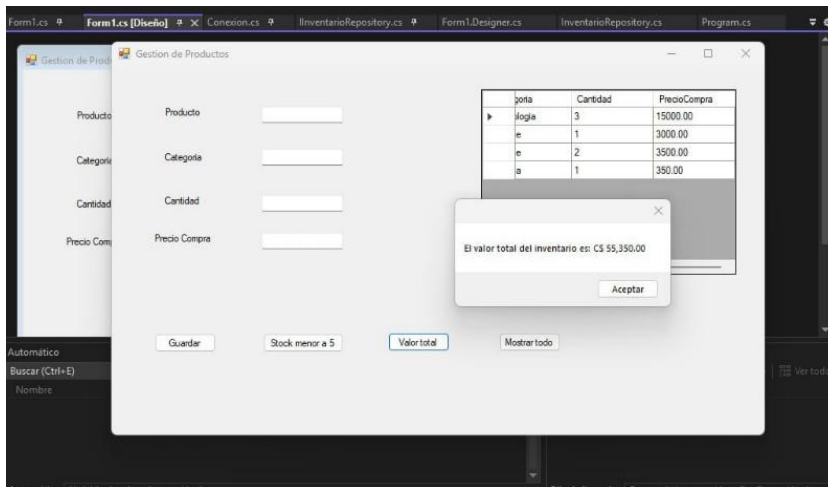
                while (reader.Read())
                {
                    lista.Add(new Inventario
                    {
                        Id = (int)reader["Id"],
                        Producto = reader["Producto"].ToString(),
                        Categoria = reader["Categoria"].ToString(),
                        Cantidad = (int)reader["Cantidad"],
                        PrecioCompra = (float)reader["PrecioCompra"]
                    });
                }
            }

            return lista;
        }
    }
}

```

Programación del repositorio

código de la clase InventarioRepository.cs. Aquí se implementa la conexión con SQL Server usando SqlConnection, además de los comandos SQL necesarios para insertar productos, consultar registros y trabajar con la tabla Inventario.

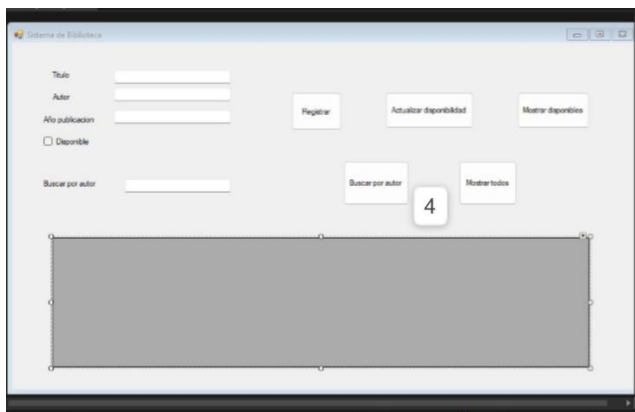


Prueba del valor total del inventario

En esta captura se muestra el programa en ejecución. Al presionar el botón Valor total, el sistema calcula el valor acumulado del inventario multiplicando la cantidad por el precio de compra de cada producto. El resultado se muestra en un mensaje, confirmando que el programa está leyendo correctamente los datos guardados en SQL Server.

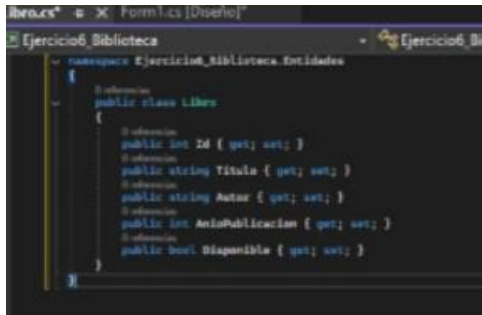
////////////////////////////////////

6) Ejercicio



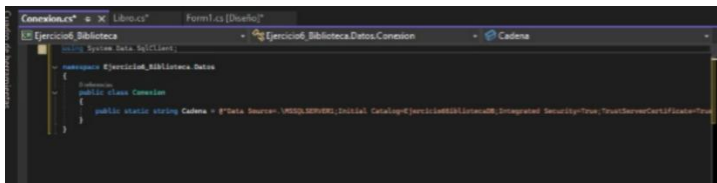
Diseño del formulario del sistema de biblioteca

diseño del formulario del sistema de biblioteca. Se agregaron los campos para ingresar Título, Autor y Año de publicación, además del CheckBox para indicar si el libro está disponible. También se incorporaron botones para registrar, actualizar disponibilidad, mostrar disponibles, buscar por autor y mostrar todos los libros, junto con un DataGridView para visualizar los registros.



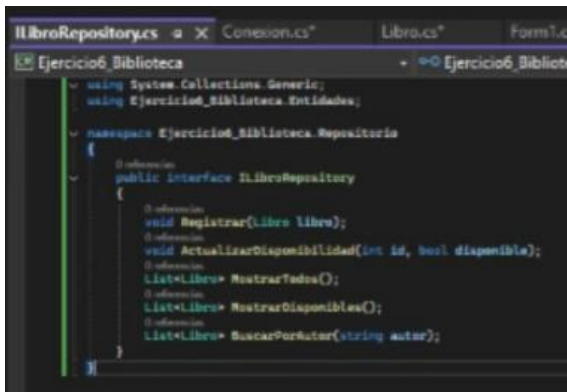
Creación de la entidad Libro

En la clase Libro.cs, ubicada dentro de la carpeta Entidades. Esta clase representa la estructura de los datos del libro, incluyendo las propiedades Id, Título, Autor, AnioPublicacion y Disponible, que corresponden a los campos de la tabla en la base de datos.



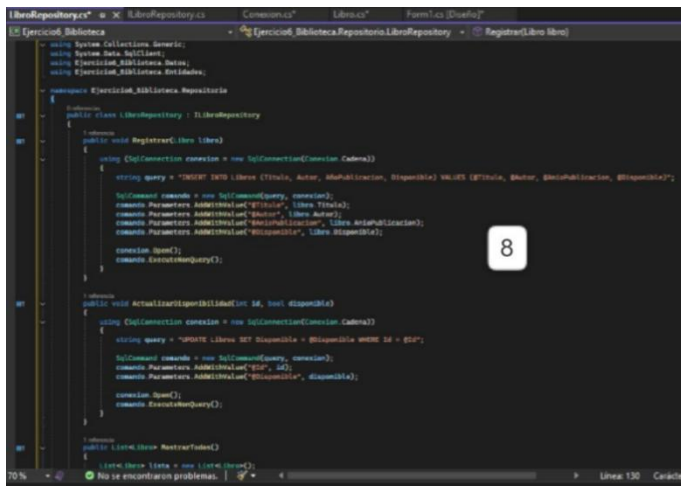
Creación de la clase Conexión

clase Conexion.cs, ubicada dentro de la carpeta Datos. Esta clase contiene la cadena de conexión hacia SQL Server, permitiendo que la aplicación pueda comunicarse con la base de datos Ejercicio6BibliotecaDB.



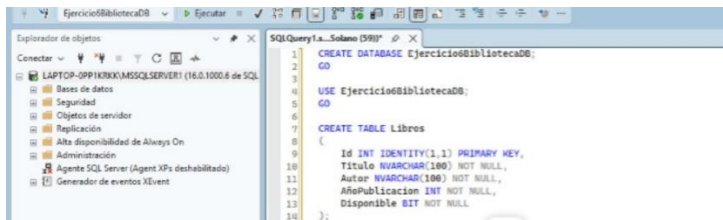
Creación de la interfaz del repositorio

se observa la interfaz ILibroRepository.cs, ubicada en la carpeta Repositorio. En ella se declararon los métodos principales del sistema, como registrar libros, actualizar disponibilidad, mostrar todos los libros, mostrar solo los disponibles y buscar libros por autor, aplicando el patrón Repository.



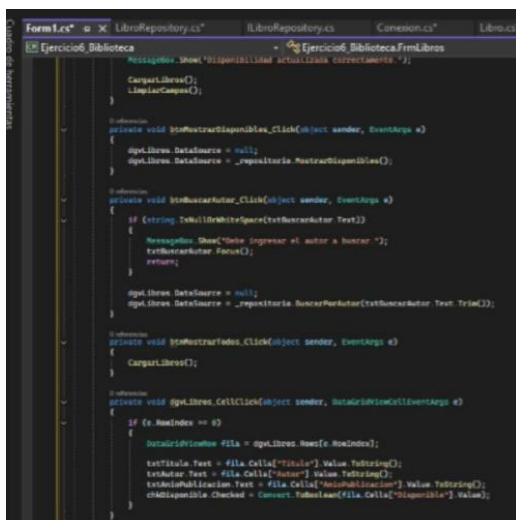
Programación del repositorio de libros

se muestra parte del código de la clase LibroRepository.cs. Esta clase implementa los métodos definidos en la interfaz y realiza las operaciones de registro, actualización de disponibilidad, consulta general, consulta de libros disponibles y búsqueda por autor mediante ADO.NET.



Creación de la base de datos y tabla Libros

En esta captura se muestra SQL Server Management Studio con el script utilizado para crear la base de datos Ejercicio6BibliotecaDB y la tabla Libros. La tabla contiene los campos Id, Titulo, Autor, AñoPublicacion y Disponible. También se observa que la consulta fue ejecutada correctamente.



Programación del formulario y funciones principales

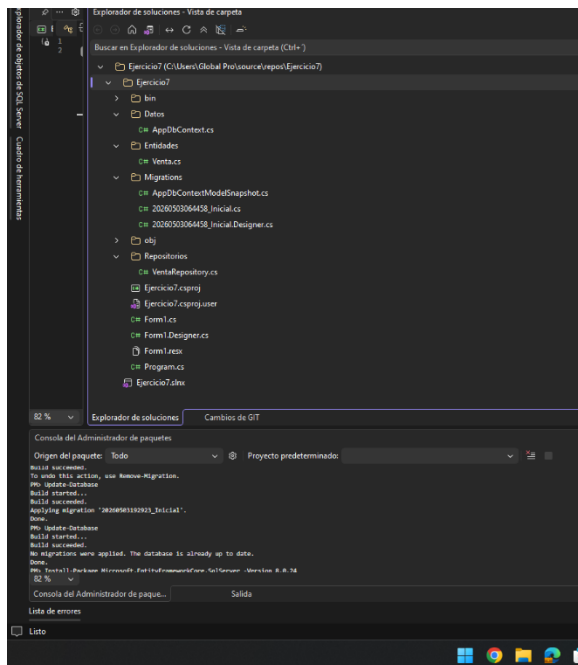
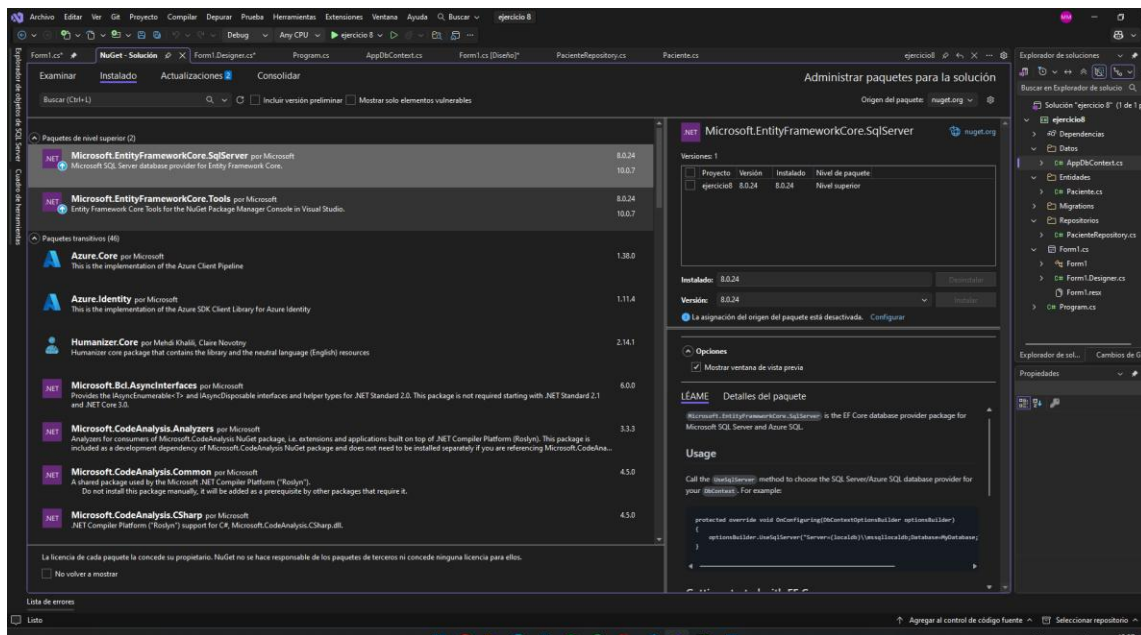
se observa parte del código del formulario FrmLibros. En este archivo se programaron las funciones principales del sistema, incluyendo la validación de campos, el registro de libros, la actualización de disponibilidad, la búsqueda por autor, la visualización de libros disponibles y la carga de todos los registros.

[illegible]

7. Ejercicio

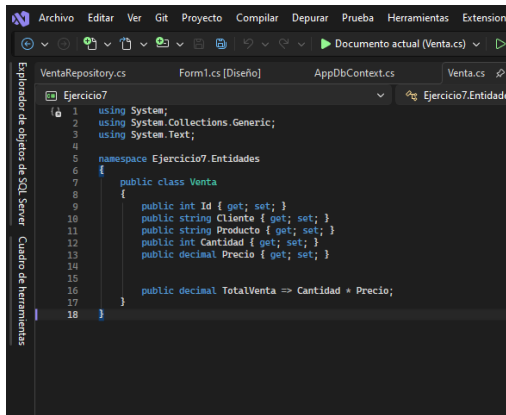
. Gestión de Dependencias (NuGet)

Se configuró el proyecto instalando los paquetes de Entity Framework Core para SQL Server, asegurando la compatibilidad con el motor de base de datos.



2. Arquitectura de Capas del Proyecto

Se organizó la solución en carpetas lógicas (Entidades, Datos, Repositorios) para separar la interfaz de la lógica de acceso a datos.



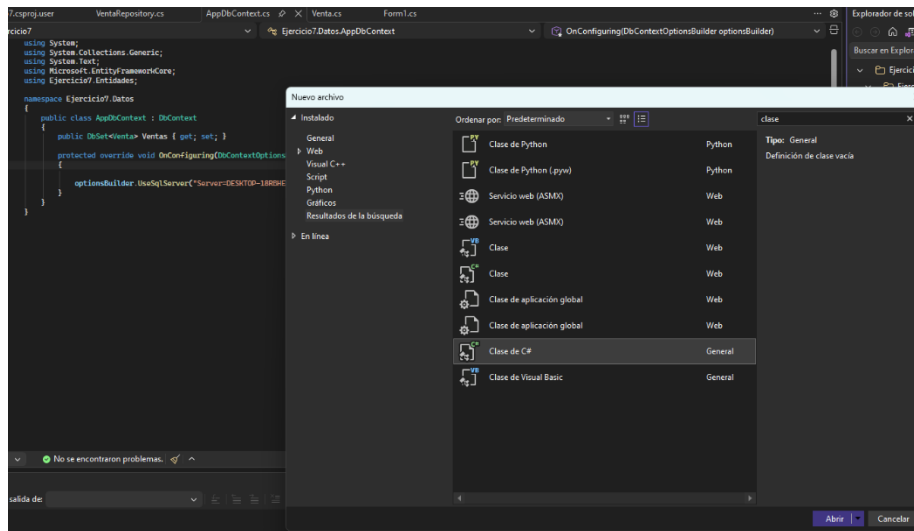
```
1 using System;
2 using System.Collections.Generic;
3 using System.Text;
4 namespace Ejercicio7.Entidades
5 {
6     public class Venta
7     {
8         public int Id { get; set; }
9         public string Cliente { get; set; }
10        public string Producto { get; set; }
11        public int Cantidad { get; set; }
12        public decimal Precio { get; set; }
13
14        public decimal TotalVenta => Cantidad * Precio;
15    }
16 }
17
18 }
```

3. Modelado de Datos (Clase Producto)

Se definió la entidad 'Venta' incluyendo propiedades para Nombre, Precio y Stock, junto con su respectiva clave primaria.

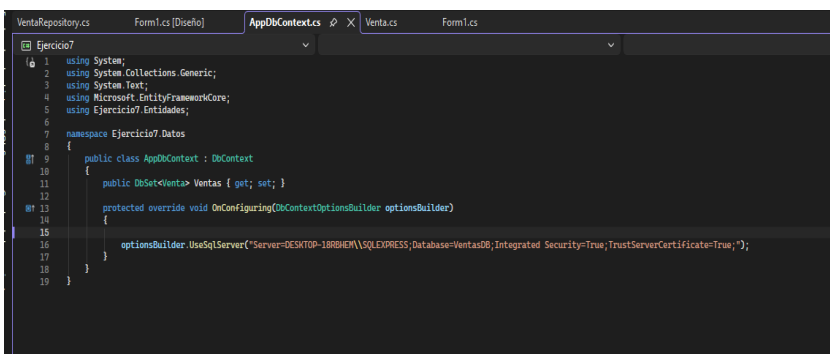
4. Agregamos una clase en la caperta recién creada

Aquí se agrega una clase la cual tendrá importancia en los siguientes pasos



5. Configuración del Contexto (AppDbContext)

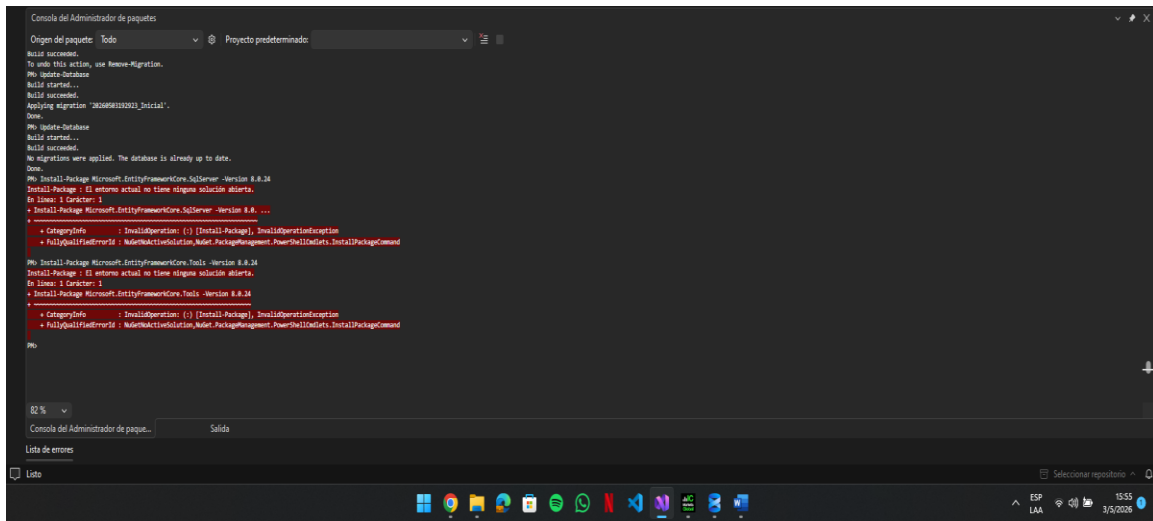
Se estableció la conexión con SQL Server LocalDB bajo el nombre 'InventarioDB', configurando el mapeo de la tabla de productos.



```
1 using System;
2 using System.Collections.Generic;
3 using System.Text;
4 using Microsoft.EntityFrameworkCore;
5 using Ejercicio7.Entidades;
6
7 namespace Ejercicio7.Datos
8 {
9     public class AppDbContext : DbContext
10     {
11         public DbSet<Venta> Ventas { get; set; }
12
13         protected override void OnModelCreating(DbContextOptionsBuilder optionsBuilder)
14         {
15             optionsBuilder.UseSqlServer("Server=DESKTOP-188BHEM\\SQLSERVER;Database=VentasDB;Integrated Security=True;TrustServerCertificate=True;");
16         }
17     }
18 }
19 }
```

7. Proceso de Migración Code-First

Se utilizaron los comandos de la consola de administración de paquetes para generar la estructura de la base de datos a partir de las clases de C#.



8. Diseño de la Interfaz de Inventario

Construcción del formulario de escritorio con controles para el ingreso de productos y botones para la ejecución de filtros de stock.

Form1

Id	Cliente	Producto	Cantidad
1	Mauricio	LLanta	1
2	Eddy	llanta	2

Cliente:

Producto:

Cantidad:

Precio:

Guardar

Total Venta Actual: \$224.00

Total General: \$346.00

Producto Top: LLanta

9. Ejecución y Registro de Inventario

Prueba de funcionamiento realizando la carga de productos iniciales y validando que se almacenen correctamente en el sistema.

Form1

Cliente
Producto
Cantidad
Precio

Guardar

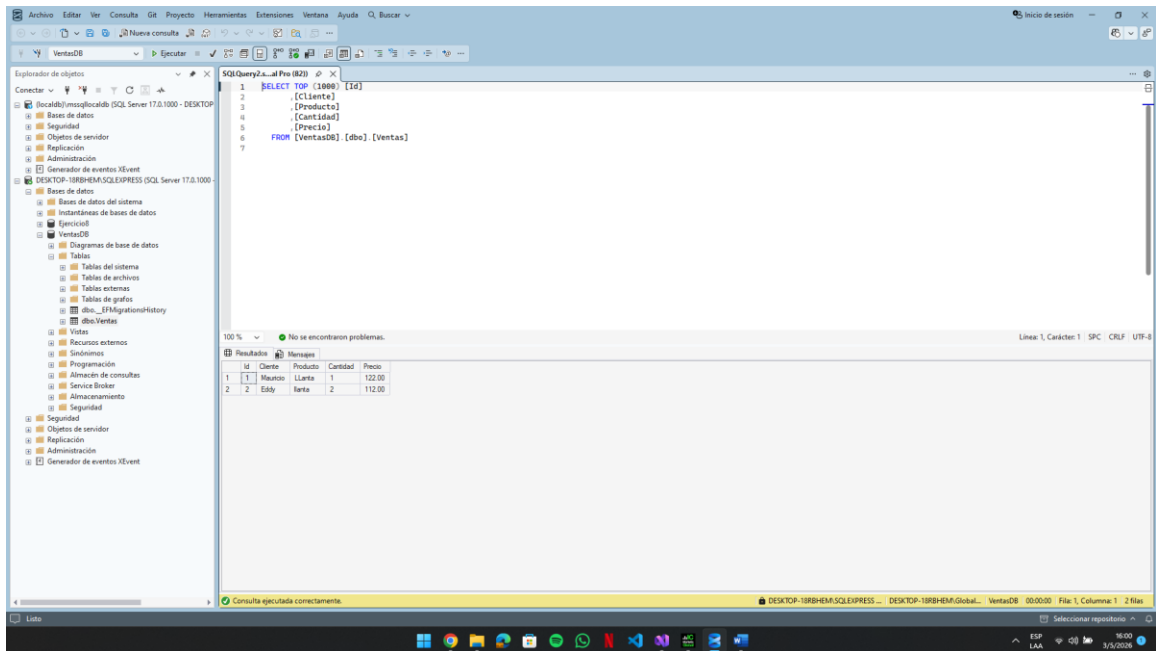
id	Cliente	Producto	Cantidad
1	Mauricio	LLanta	1
2	Eddy	Renta	2

Total Venta Actual: \$234.00
Total General: \$346.00
Producto Top: LLanta

Windows taskbar: 15:37 1/5/2024

10. Comprobación en SQL Server Management Studio

Inspección final de la base de datos 'InventarioDB' para verificar que la información es persistente y coherente.

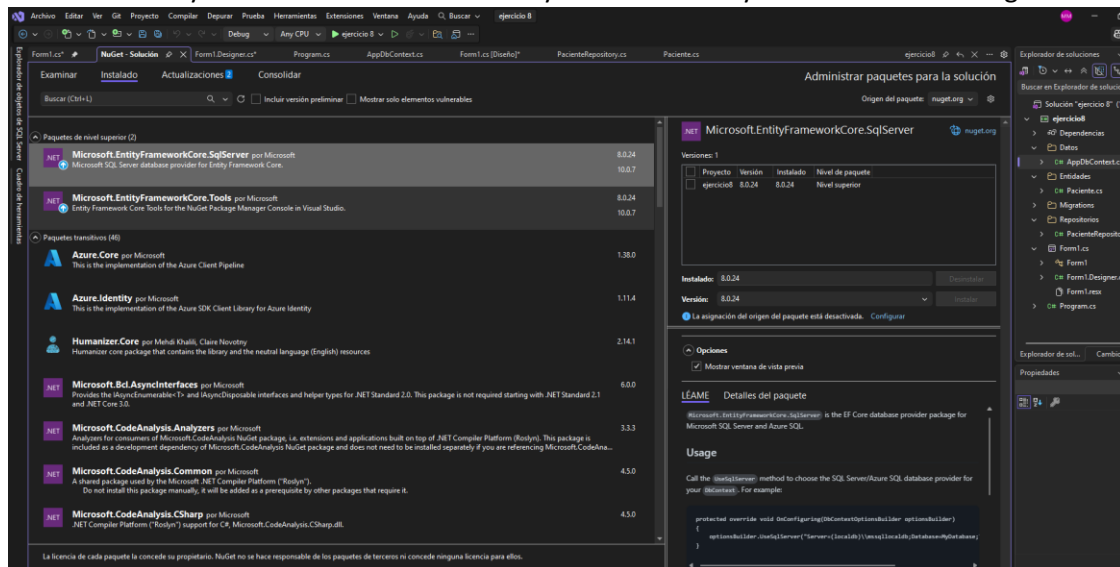


////////////////////////////////////

8) Ejercicio

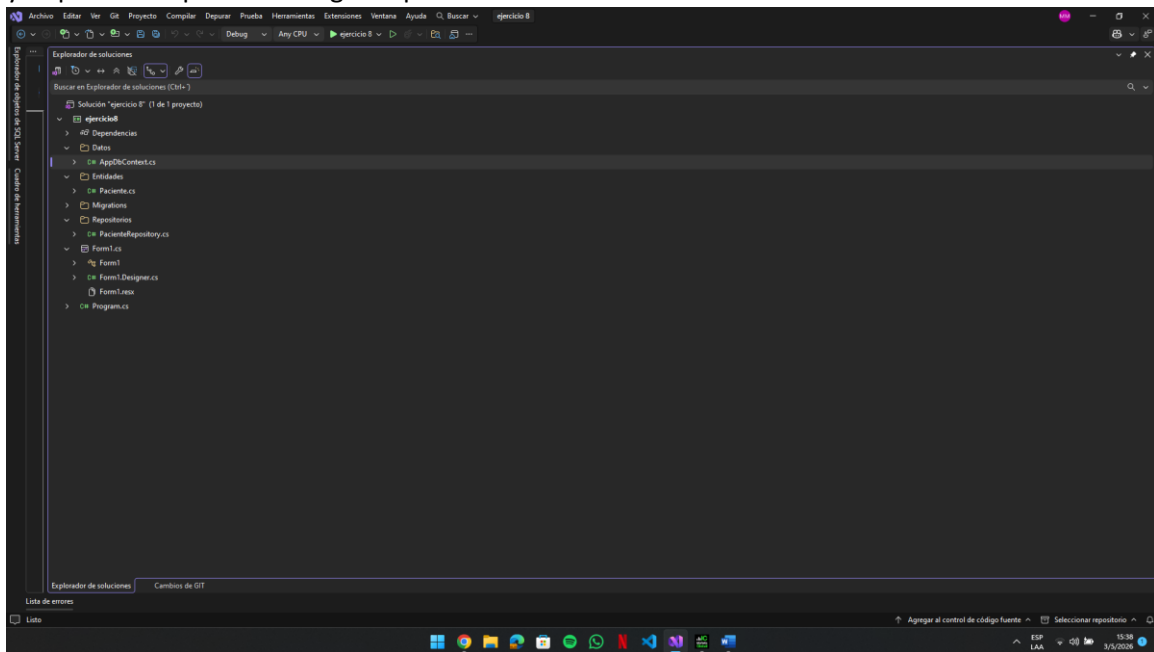
1. Configuración de Dependencias (NuGet)

Se instalaron los paquetes necesarios para trabajar con SQL Server y Entity Framework Core: Microsoft.EntityFrameworkCore, SQL Server y Microsoft.EntityFrameworkCore.Design.



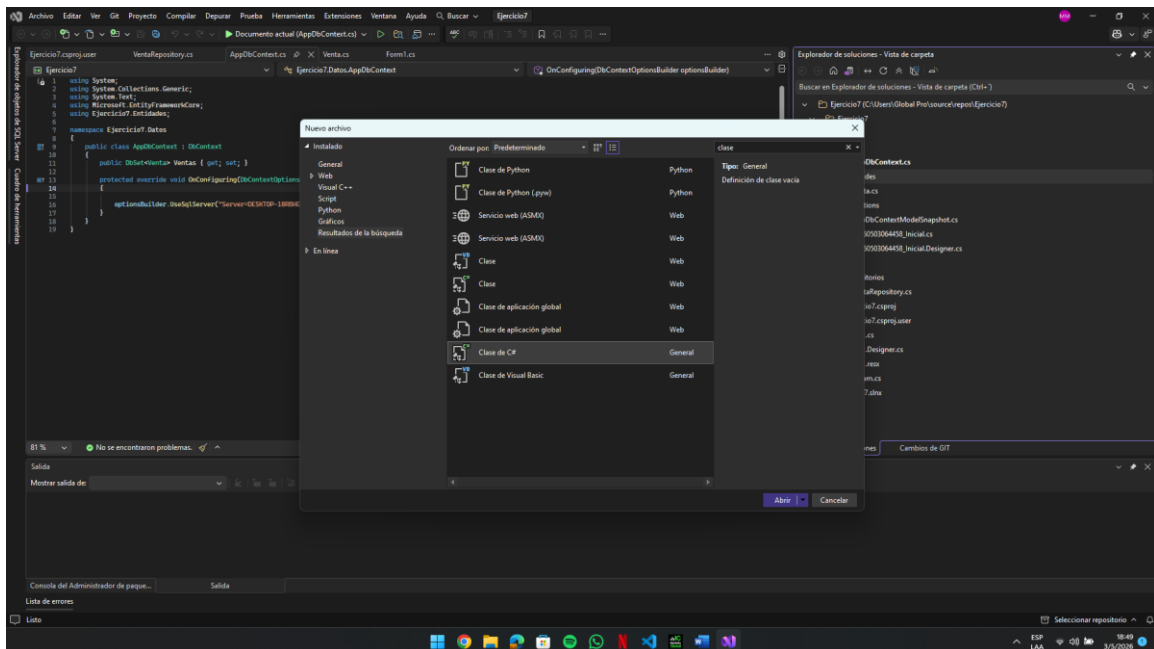
2. Estructura de Carpetas del Proyecto

Se organizó el proyecto siguiendo el patrón de repositorio, creando carpetas para Entidades, Datos y Repositorios para un código limpio.



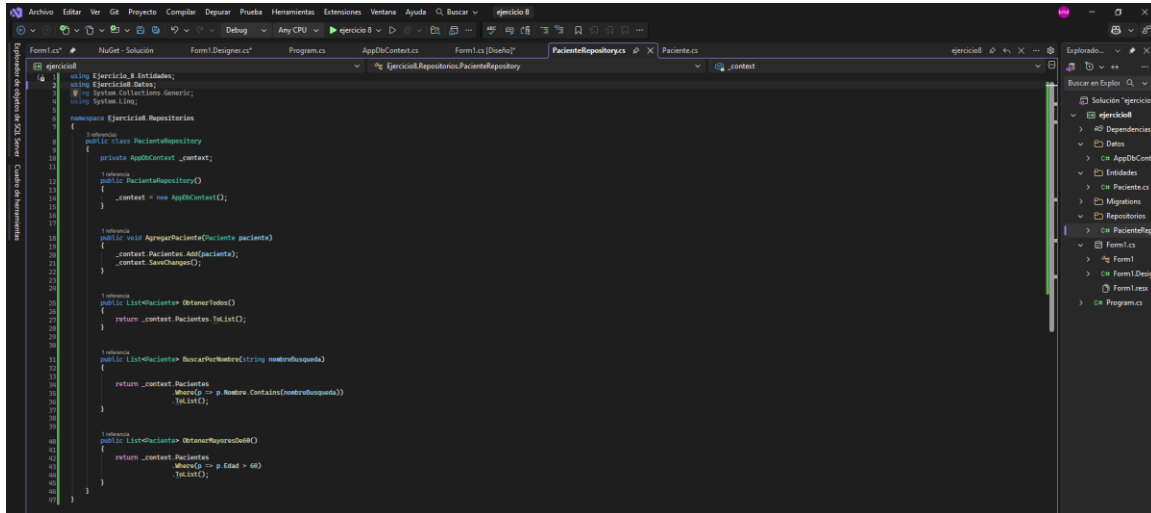
3. Agregamos una clase en la carpeta recién creada

Aquí se agrega una clase la cual tendrá importancia en los siguientes pasos



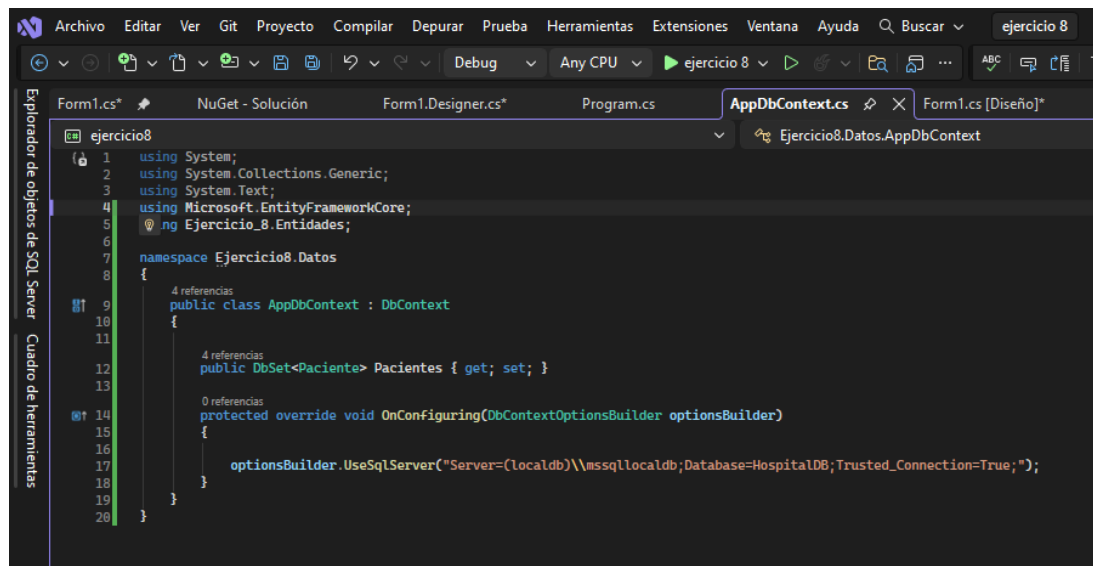
4. Definición del Modelo (Clase Paciente)

Se creó la clase 'Paciente' con sus propiedades automáticas (Id, Nombre, Edad, Diagnóstico) que servirán como columnas en la tabla.



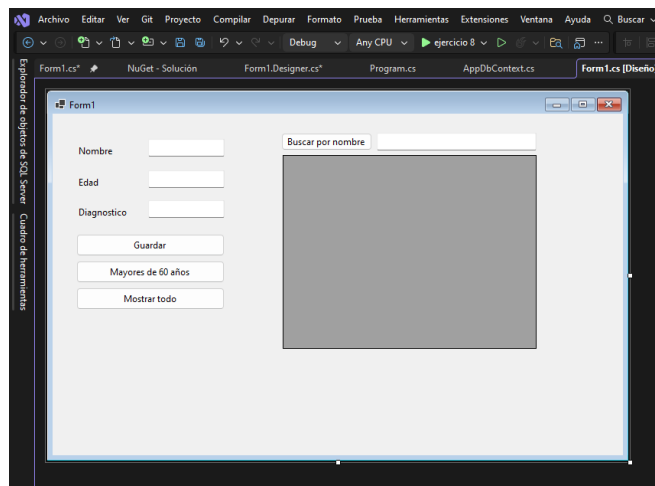
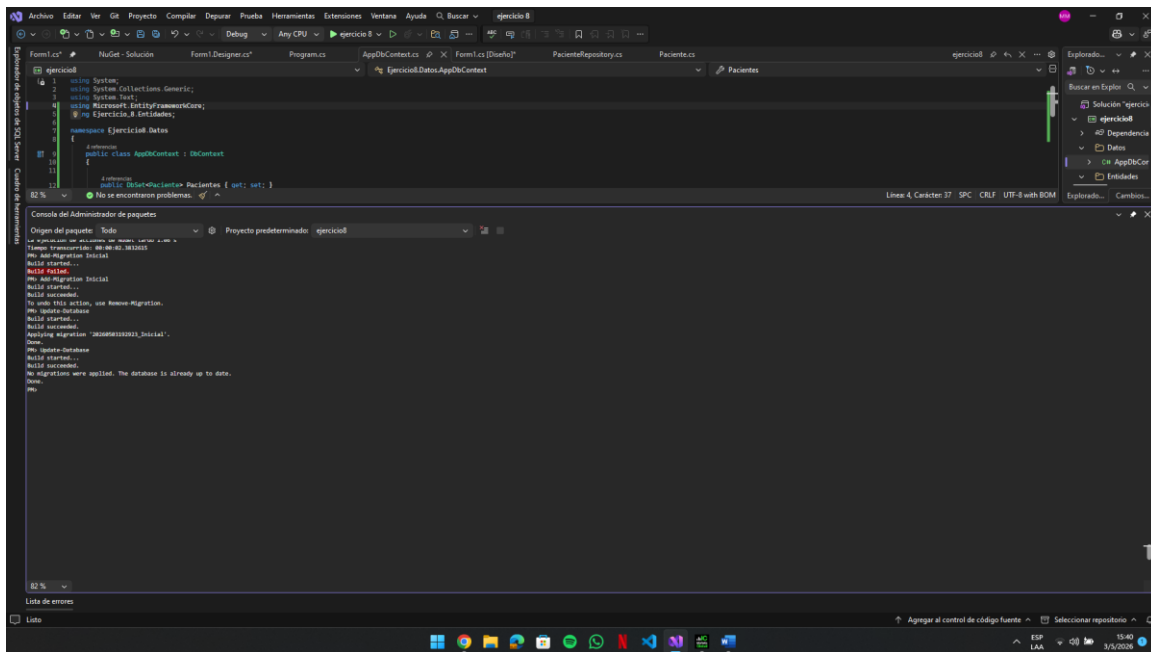
5. Configuración del DbContext

Se configuró la clase AppDbContext definiendo la cadena de conexión a SQL Server LocalDB y el DbSet para la tabla de Pacientes.



6. Aplicación de Migraciones (Code-First)

Se ejecutaron los comandos 'Add-Migration' y 'Update-Database' para transformar nuestras clases de C# en una base de datos real.



7. Diseño de Interfaz (GUI)

Se diseñó el formulario principal en WinForms utilizando `TextBox`, `Button` y un `DataGridView` para la visualización de los datos.

```

{
    private ApplicationDbContext _context;

    1 referencia
    public PacienteRepository()
    {
        _context = new ApplicationDbContext();
    }

    1 referencia
    public void AgregarPaciente(Paciente paciente)
    {
        _context.Pacientes.Add(paciente);
        _context.SaveChanges();
    }

    1 referencia
    public List<Paciente> ObtenerTodos()
    {
        return _context.Pacientes.ToList();
    }

    1 referencia
    public List<Paciente> BuscarPorNombre(string nombreBusqueda)
    {
        return _context.Pacientes
            .Where(p => p.Nombre.Contains(nombreBusqueda))
            .ToList();
    }

    1 referencia
    public List<Paciente> ObtenerMayoresDe60()
    {
        return _context.Pacientes
            .Where(p => p.Edad > 60)
            .ToList();
    }
}

```

8. Implementación de Lógica LINQ (Filtro de Edad)

Se programó el método para extraer pacientes mayores de 60 años utilizando una expresión Lambda: `.Where(p => p.Edad > 60)`.

9. Pruebas de Registro de Datos

Ejecución de la aplicación ingresando nuevos pacientes y confirmando el guardado exitoso mediante el repositorio.

	Nombre	Edad	Diagnostico
▶	Mauricio	22	Gripe
	Rosa	70	cancer
	Aura	23	rotura crane
	Eddy	99	falla motora

10. Pruebas de Búsqueda y Filtrado

Validación del buscador por nombre y del filtro de edad directamente en la interfaz de usuario con el programa en ejecución.

Id	Nombre	Edad
2	Rosa	70
4	Eddy	99

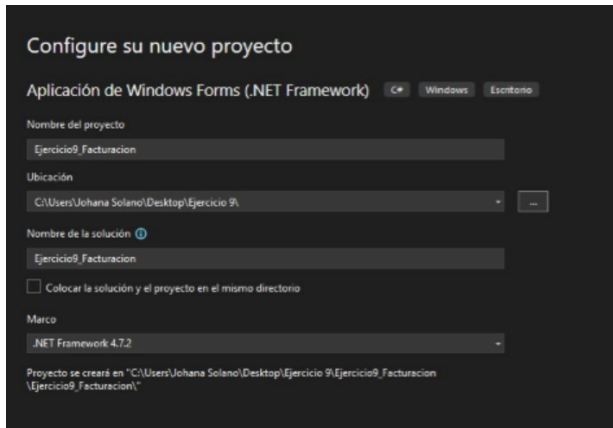
11. Verificación Final en SQL Server (SSMS)

Comprobación de que la tabla 'Pacientes' en la base de datos 'HospitalDB' contiene todos los registros guardados desde el programa.

```
SELECT TOP (1000) [Id], [Nombre], [Edad], [Diagnostico] FROM [HospitalDB].[dbo].[Pacientes]
```

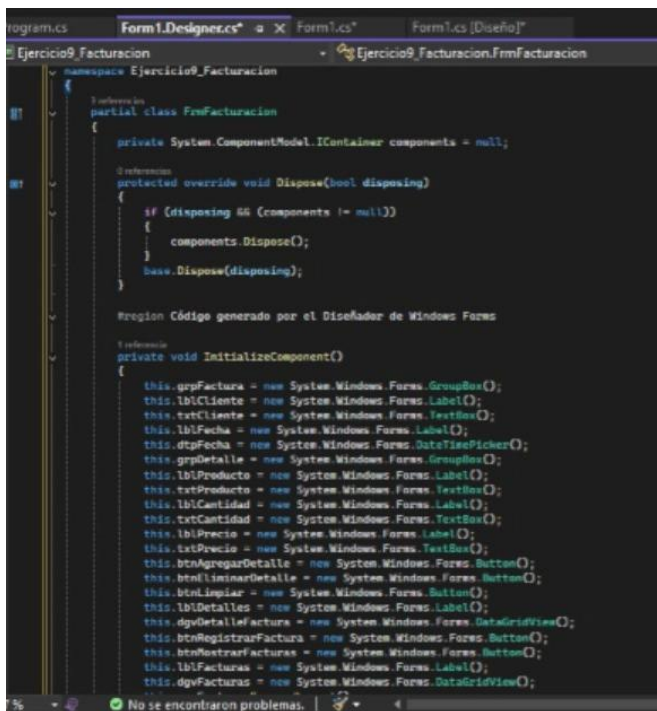
Id	Nombre	Edad	Diagnostico
1	Mauricio	22	Gripe
2	Rosa	70	cancer
3	Aura	23	rotura craneal
4	Eddy	99	falla motora de pulmones

9) Ejercicio



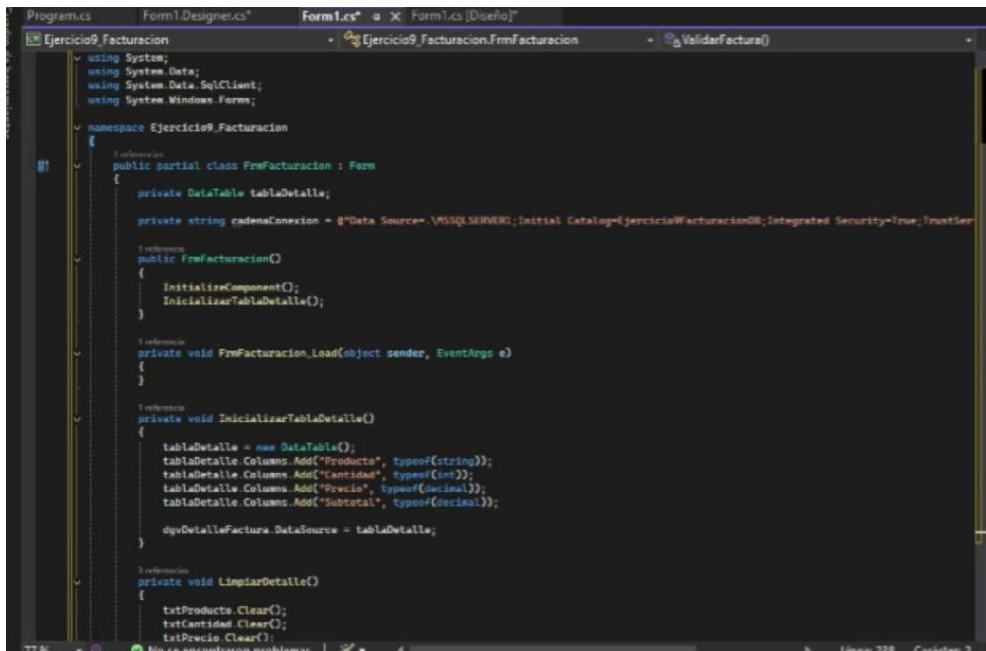
Configuración inicial del proyecto

se muestra la configuración inicial del proyecto en Visual Studio. Se seleccionó una aplicación de Windows Forms (.NET Framework), asignando como nombre del proyecto y de la solución Facturacion. También se configuró la ubicación del proyecto dentro de la carpeta correspondiente al Ejercicio 9 y se seleccionó el marco .NET Framework 4.7.2.



Estructura del formulario principal

la estructura del formulario principal del sistema de facturación. Se incorporaron los controles necesarios para ingresar los datos de la factura, como cliente y fecha, además de los campos del detalle de factura, botones de acción y tablas para mostrar los detalles agregados y las facturas registradas.



```
using System;
using System.Data;
using System.Data.SqlClient;
using System.Windows.Forms;

namespace Ejercicio9_Facturacion
{
    public partial class FrmFacturacion : Form
    {
        private DataTable tablaDetalle;

        private string cadenaConexion = @"Data Source=.\MSQLSERVER;Initial Catalog=EjercicioFacturacionDB;Integrated Security=True;TrustSer

        public FrmFacturacion()
        {
            InitializeComponent();
            InicializarTablaDetalle();
        }

        private void FrmFacturacion_Load(object sender, EventArgs e)
        {
        }

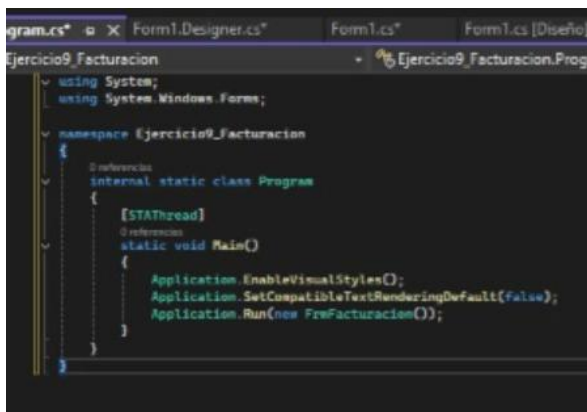
        private void InicializarTablaDetalle()
        {
            tablaDetalle = new DataTable();
            tablaDetalle.Columns.Add("Producto", typeof(string));
            tablaDetalle.Columns.Add("Cantidad", typeof(int));
            tablaDetalle.Columns.Add("Precio", typeof(decimal));
            tablaDetalle.Columns.Add("Subtotal", typeof(decimal));

            dgvDetalleFactura.DataSource = tablaDetalle;
        }

        private void LimpiarDetalle()
        {
            txtProducto.Clear();
            txtCantidad.Clear();
            txtPrecio.Clear();
        }
    }
}
```

Programación de la lógica principal del sistema

se muestra parte de la lógica principal del formulario FrmFacturacion. En este archivo se configuró la conexión con la base de datos, la tabla temporal para almacenar los detalles de la factura y los métodos necesarios para limpiar campos, validar datos y preparar la información antes de registrar la factura.

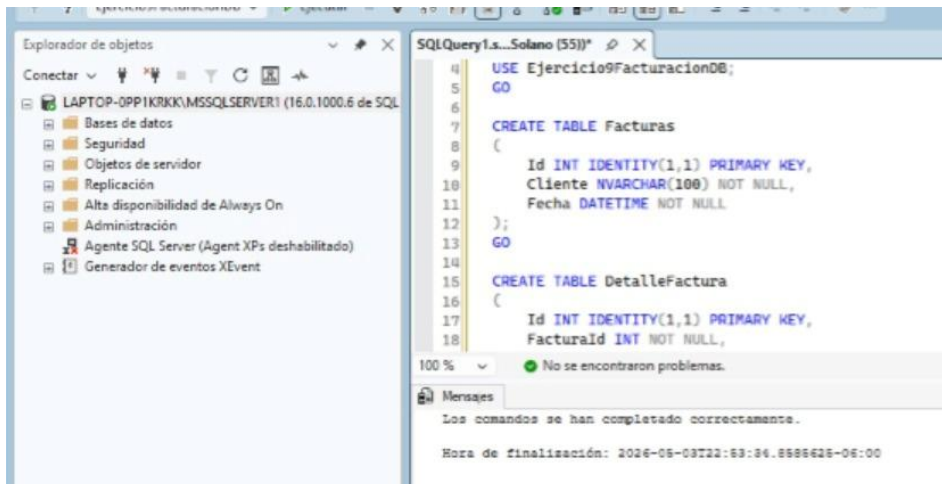


```
using System;
using System.Windows.Forms;

namespace Ejercicio9_Facturacion
{
    internal static class Program
    {
        [STAThread]
        static void Main()
        {
            Application.EnableVisualStyles();
            Application.SetCompatibleTextRenderingDefault(false);
            Application.Run(new FrmFacturacion());
        }
    }
}
```

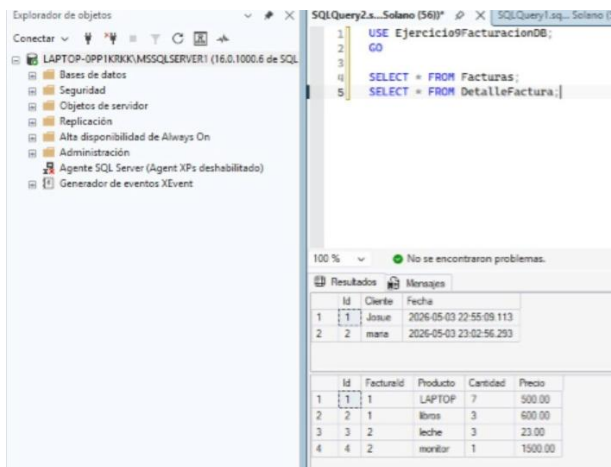
Configuración del archivo Program.cs

En esta imagen se observa el archivo Program.cs, donde se configuró el formulario principal que se ejecuta al iniciar la aplicación. La instrucción `Application.Run(new FrmFacturacion());` permite que el sistema abra directamente el formulario de facturación al ejecutar el proyecto.



Creación de la base de datos y tablas

se muestra SQL Server Management Studio con el script utilizado para crear la base de datos Ejercicio9FacturacionDB. También se crearon las tablas Facturas y DetalleFactura, estableciendo la relación entre ambas mediante el campo Facturald. La consulta fue ejecutada correctamente.



Verificación de registros en las tablas

se muestra la verificación de los datos almacenados en SQL Server. Se ejecutaron consultas para visualizar los registros guardados en las tablas Facturas y DetalleFactura, comprobando que las facturas y sus respectivos productos fueron registrados correctamente en la base de datos.

```

1 SELECT
2     F.Id AS FacturaId,
3     F.Cliente,
4     F.Fecha,
5     D.Producto,
6     D.Cantidad,
7     D.Precio,
8     D.Cantidad * D.Precio AS Subtotal
9 FROM Facturas F
10 INNER JOIN DetalleFactura D ON F.Id = D.FacturaId;

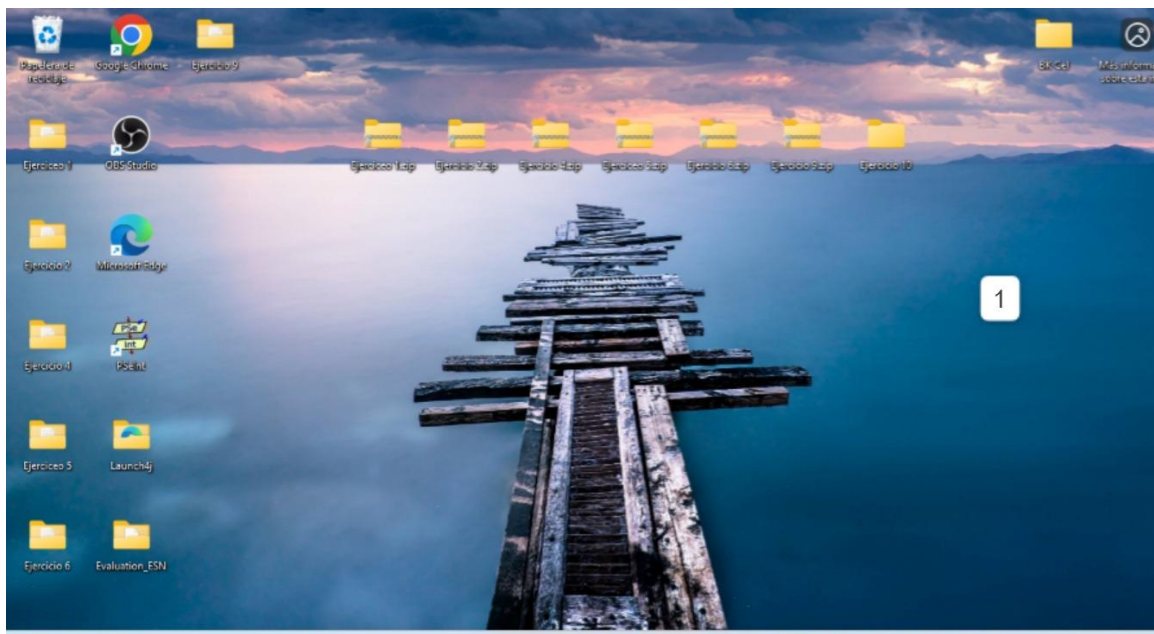
```

	FacturaId	Cliente	Fecha	Producto	Cantidad	Precio	Subtotal
1	1	Josue	2026-05-03 22:55:09.113	LAPTOP	7	500.00	3500.00
2	1	Josue	2026-05-03 22:55:09.113	libros	3	600.00	1800.00
3	2	maria	2026-05-03 23:02:56.293	leche	3	23.00	69.00
4	2	maria	2026-05-03 23:02:56.293	monitor	1	1500.00	1500.00

Consulta de facturas con sus detalles

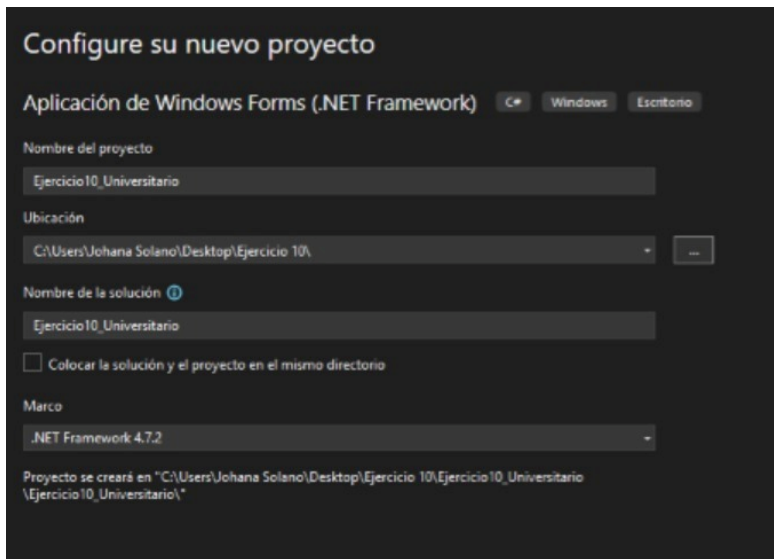
se observa una consulta con unión entre las tablas Facturas y DetalleFactura. Esta consulta permite mostrar cada factura junto con el cliente, fecha, producto, cantidad, precio y subtotal, comprobando que la relación entre ambas tablas funciona correctamente.

10) Ejercicio



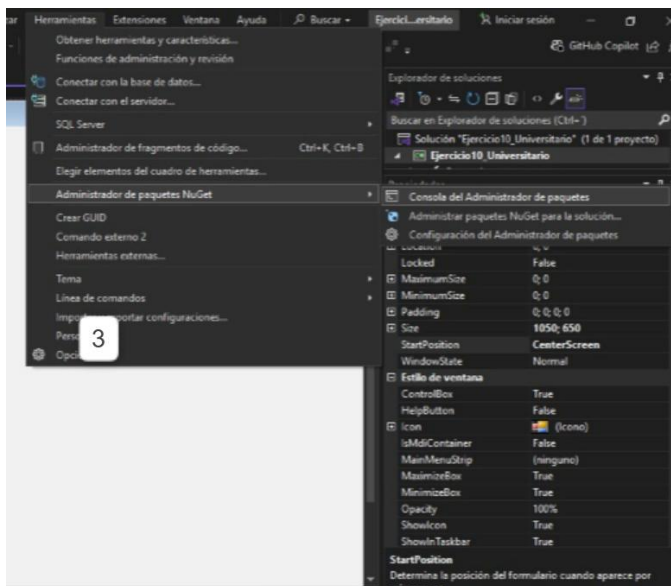
Carpeta del ejercicio

Se muestra la carpeta correspondiente al Ejercicio 10, creada en el escritorio para organizar los archivos del sistema universitario.



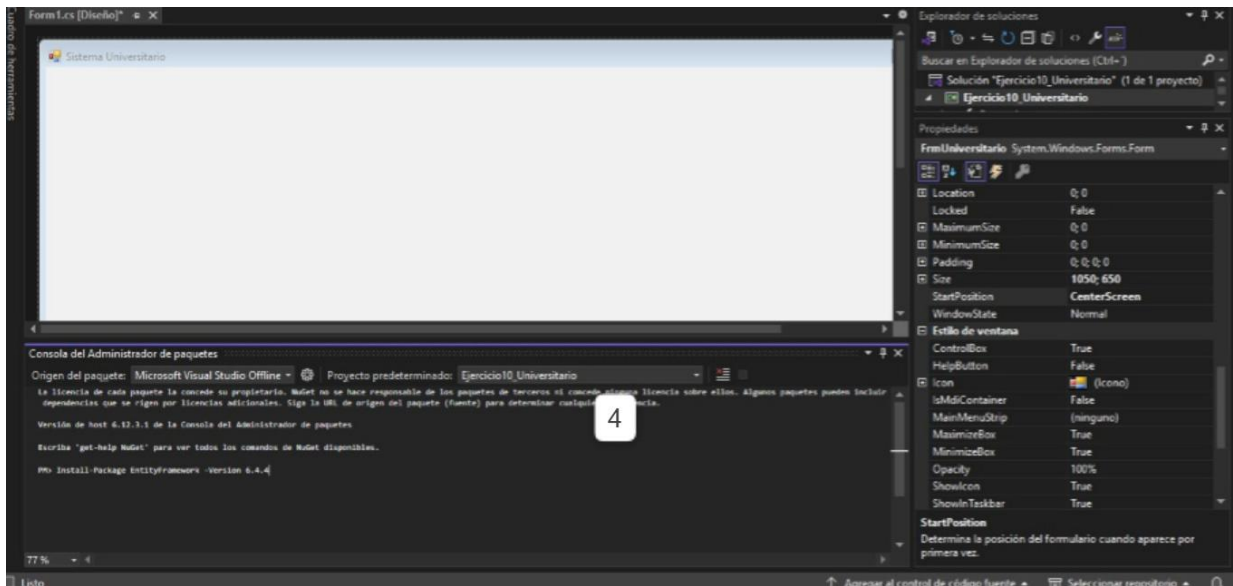
Configuración inicial del proyecto

Se observa la creación del proyecto Ejercicio10_Universitario en Visual Studio, utilizando una aplicación de Windows Forms con .NET Framework 4.7.2.



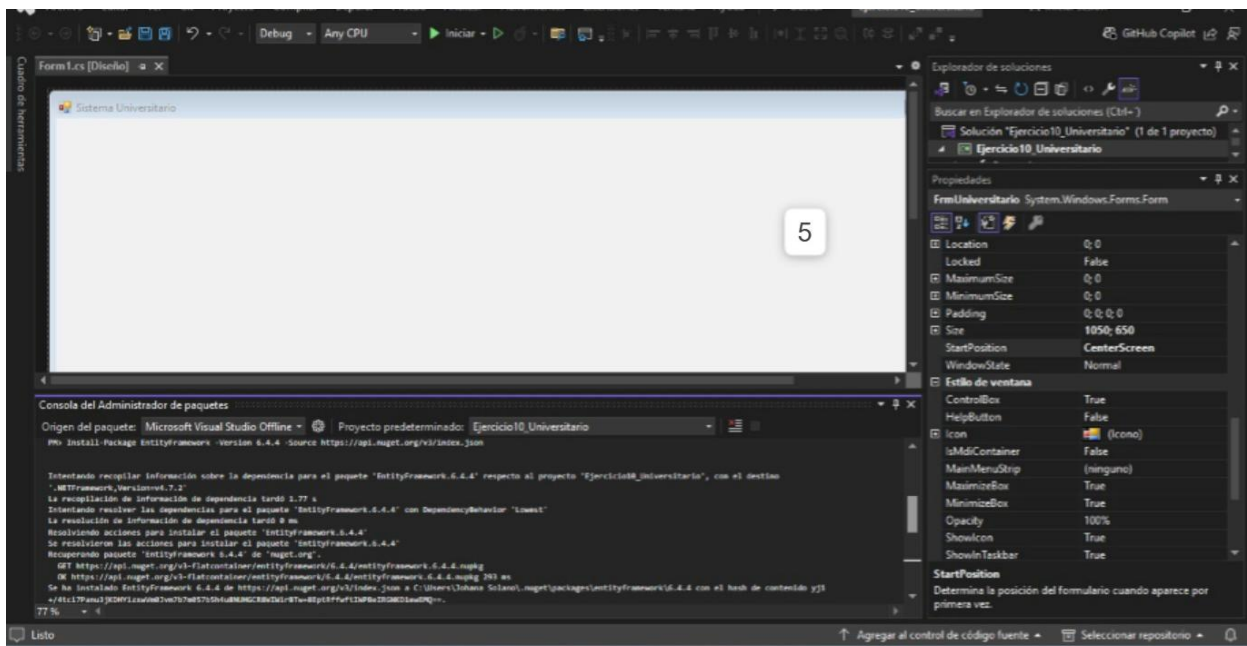
Acceso al administrador de paquetes NuGet

Se muestra el acceso a la consola del administrador de paquetes NuGet desde Visual Studio, necesaria para instalar Entity Framework.



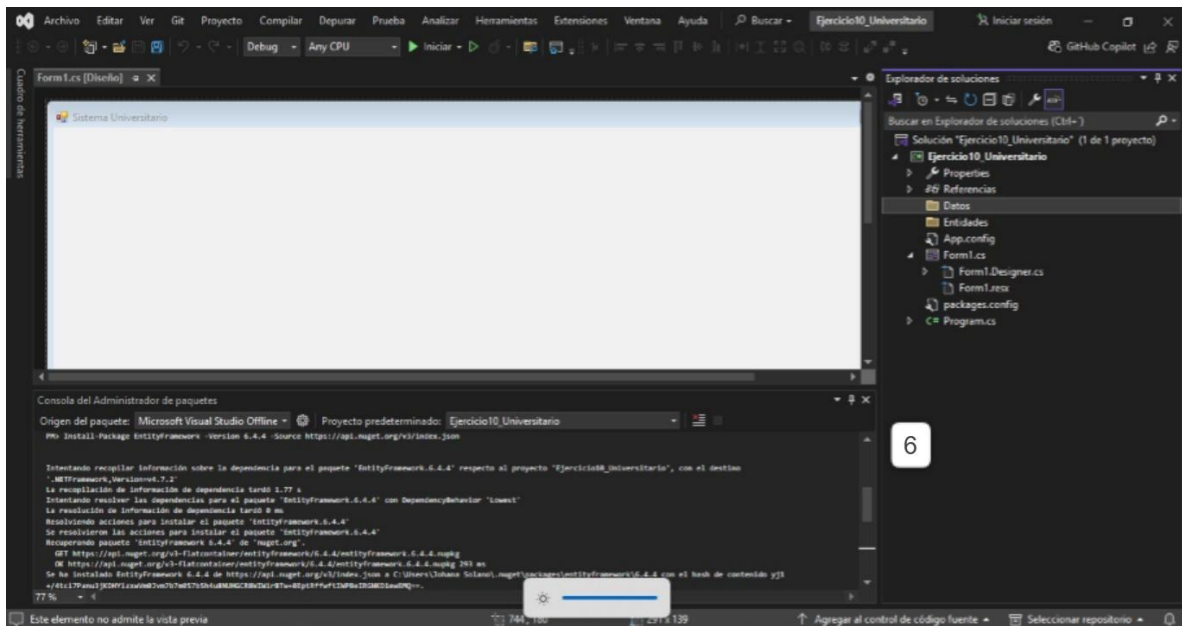
Consola del administrador de paquetes

Se observa la consola de NuGet abierta dentro de Visual Studio, donde se prepara la instalación del paquete requerido para trabajar con Entity Framework.



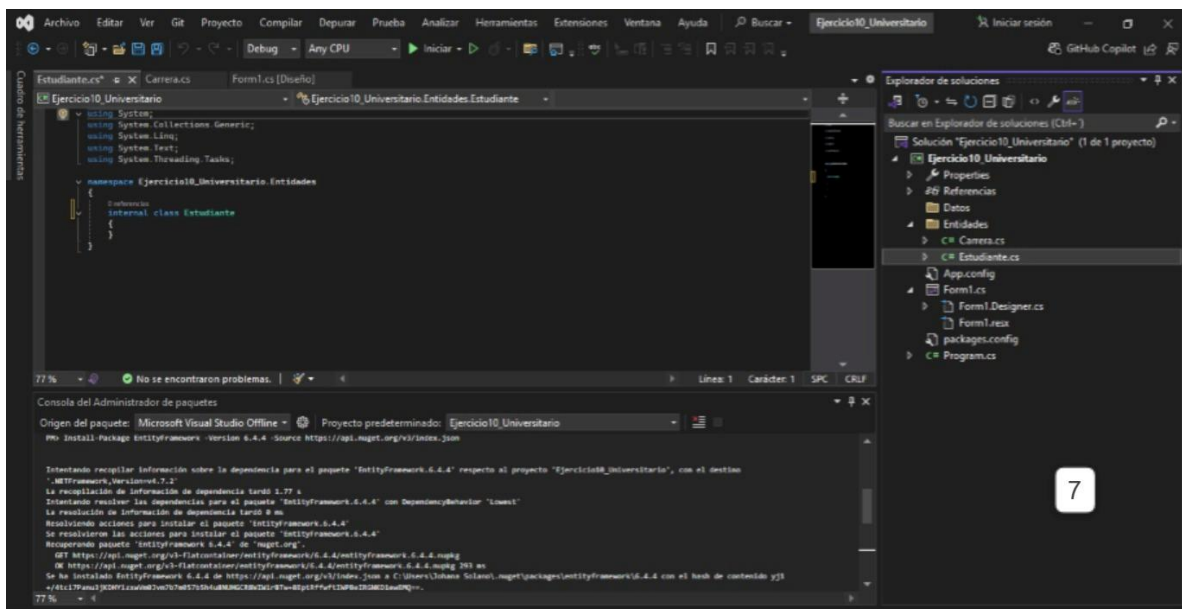
Instalación de Entity Framework

Se muestra el proceso de instalación de EntityFramework 6.4.4, utilizado para manejar la conexión y relación entre las tablas del sistema universitario.



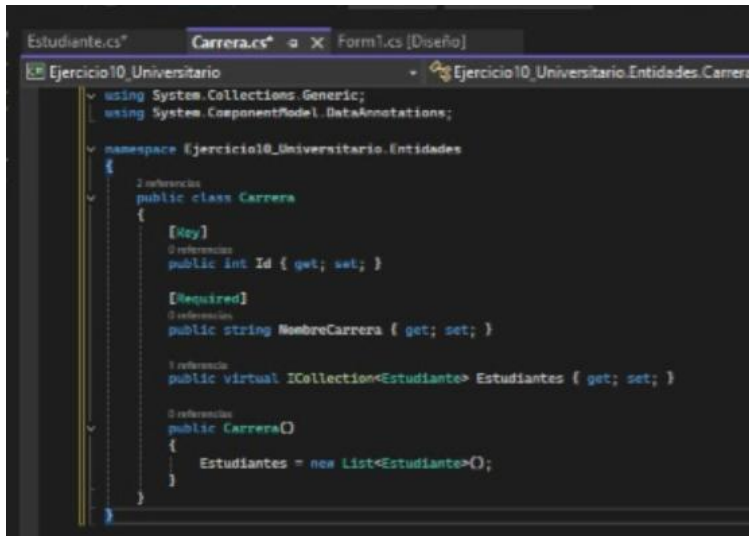
Organización del proyecto

Se observa la estructura del proyecto con las carpetas Datos y Entidades, utilizadas para separar las clases principales del sistema.



Descartar

Esta imagen se puede descartar porque muestra la clase Estudiante.cs todavía sin completar.



```
using System.Collections.Generic;
using System.ComponentModel.DataAnnotations;

namespace Ejercicio10_Universitario.Entidades
{
    public class Carrera
    {
        [Key]
        public int Id { get; set; }

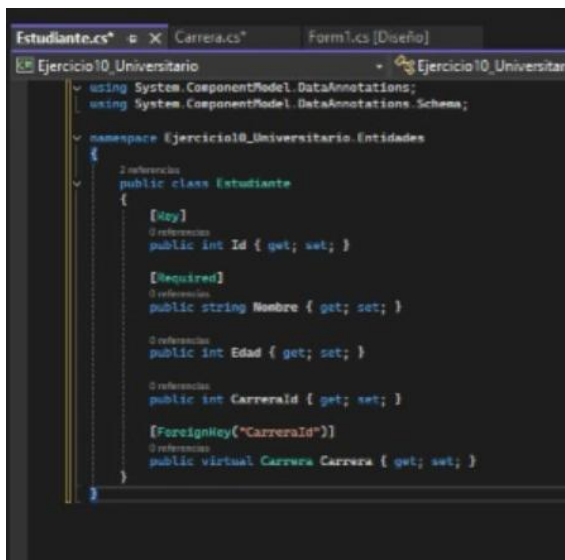
        [Required]
        public string NombreCarrera { get; set; }

        public virtual ICollection<Estudiante> Estudiantes { get; set; }

        public Carrera()
        {
            Estudiantes = new List<Estudiante>();
        }
    }
}
```

Clase Carrera

Se muestra la clase Carrera, donde se definen los datos principales de una carrera y su relación con los estudiantes.



```
using System.ComponentModel.DataAnnotations;
using System.ComponentModel.DataAnnotations.Schema;

namespace Ejercicio10_Universitario.Entidades
{
    public class Estudiante
    {
        [Key]
        public int Id { get; set; }

        [Required]
        public string Nombre { get; set; }

        public int Edad { get; set; }

        public int CarreraId { get; set; }

        [ForeignKey("CarreraId")]
        public virtual Carrera Carrera { get; set; }
    }
}
```

Clase Estudiante

Se observa la clase Estudiante, donde se definen los campos del estudiante y la relación con la carrera mediante CarreraId.

```

using System.Data.Entity;
using Ejercicio10_Universitario.Entidades;

namespace Ejercicio10_Universitario.Datos
{
    public class UniversidadContext : DbContext
    {
        public UniversidadContext() : base("name=ConexionUniversitario")
        {
        }

        public DbSet<Carrera> Carreras { get; set; }
        public DbSet<Estudiante> Estudiantes { get; set; }

        protected override void OnModelCreating(DbModelBuilder modelBuilder)
        {
            modelBuilder.Entity<Carrera>().ToTable("Carreras");
            modelBuilder.Entity<Estudiante>().ToTable("Estudiantes");

            modelBuilder.Entity<Carrera>()
                .HasMany(c => c.Estudiantes)
                .WithRequired(e => e.Carrera)
                .HasForeignKey(e => e.CarreraId);

            base.OnModelCreating(modelBuilder);
        }
    }
}

```

Contexto de base de datos

Se muestra la clase UniversidadContext, encargada de representar la conexión con la base de datos y la relación entre las tablas Carreras y Estudiantes.

```

using System;
using System.Data.Entity;
using System.Linq;
using System.Windows.Forms;
using Ejercicio10_Universitario.Datos;
using Ejercicio10_Universitario.Entidades;

namespace Ejercicio10_Universitario
{
    public partial class FrmUniversitario : Form
    {
        public FrmUniversitario()
        {
            InitializeComponent();
        }

        private void FrmUniversitario_Load(object sender, EventArgs e)
        {
            Database.SetInitializer(new CreateDatabaseIfNotExists<UniversidadContext>());
            CargarCarrerasCombo();
            MostrarCarreras();
            MostrarEstudiantes();
        }

        private void LimpiarCarrera()
        {
            txtNombreCarrera.Clear();
            txtNombreCarrera.Focus();
        }

        private void LimpiarEstudiante()
        {
            txtNombreEstudiante.Clear();
            txtIdad.Clear();

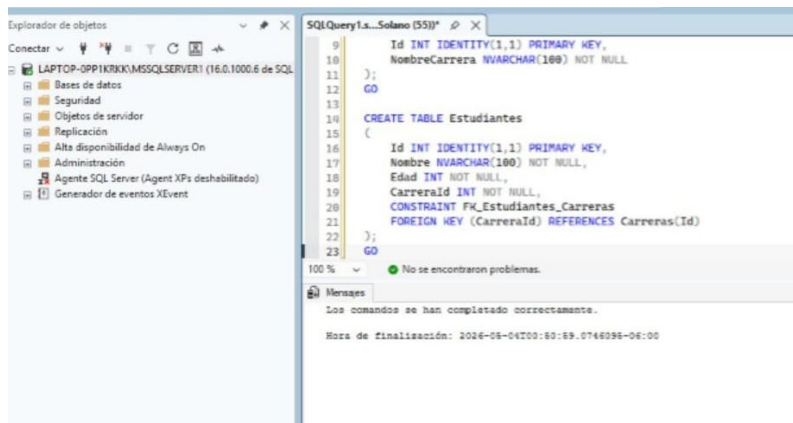
            if (cmbCarrera.Items.Count > 0)
            {
                cmbCarrera.SelectedIndex = 0;
            }

            txtNombreEstudiante.Focus();
        }
    }
}

```

Lógica principal del formulario

Se observa parte del archivo principal del formulario, donde se programan las funciones para cargar carreras, mostrar registros, limpiar campos y manejar la información del sistema.



Creación de base de datos y tablas

Se muestra SQL Server Management Studio con el script utilizado para crear la base de datos Ejercicio10UniversitarioDB y las tablas Carreras y Estudiantes, incluyendo la relación entre ambas.